

透過強化學習實現行動機器人的虛實轉移：從 NVIDIA Isaac Sim 到 Gazebo 和真實 ROS 2 機器人

Sahar Salimpour, Jorge Peña-Queralta, Diego Paez-Granados, Jukka Heikkonen, and Tomi Westerlund

Sahar Salimpour、Jorge Peña-Queralta、Diego Paez-Granados、Jukka Heikkonen 和 Tomi Westerlund

Abstract

摘要

Unprecedented agility and dexterous manipulation have been demonstrated with controllers based on deep reinforcement learning (RL), with a significant impact on legged and humanoid robots. Modern tooling and simulation platforms, such as NVIDIA Isaac Sim, have been enabling such advances. This article focuses on demonstrating the applications of Isaac in local planning and obstacle avoidance as one of the most fundamental ways in which a mobile robot interacts with its environments. Although there is extensive research on proprioception-based RL policies, the article highlights less standardized and reproducible approaches to exteroception. At the same time, the article aims to provide a base framework for end-to-end local navigation policies and how a custom robot can be trained in such simulation environment. We benchmark end-to-end policies with the state-of-the-art Nav2, navigation stack in Robot Operating System (ROS). We also cover the sim-to-real transfer process by demonstrating zero-shot transferability of policies trained in the Isaac simulator to real-world robots. This is further evidenced by the tests with different simulated robots, which show the generalization of the learned policy. Finally, the benchmarks demonstrate comparable performance to Nav2, opening the door to quick deployment of state-of-the-art end-to-end local planners for custom robot platforms, but importantly furthering the possibilities by expanding the state and action spaces or task definitions for more complex missions. Overall, with this article we introduce the most important steps, and aspects to consider, in deploying RL policies for local path planning and obstacle avoidance with Isaac Sim training, Gazebo testing, and ROS 2 for real-time inference in real robots. The code is available at <https://github.com/sahars93/RL-Navigation>

基於深度強化學習 (RL) 的控制器已展現出前所未有的敏捷性和靈巧操作能力，對足式和人形機器人產生了重大影響。NVIDIA Isaac Sim 等現代工具和模擬平台促成了這些進步。本文著重於展示 Isaac 在局部路徑規劃和避障方面的應用，這是行動機器人與其環境互動最基本的方式之一。儘管關於基於本體感覺的 RL 策略已有廣泛研究，但本文強調了外感受方面較不標準化和可重現的方法。同時，本文旨在為端到端局部導航策略以及如何在此種模擬環境中訓練客製化機器人提供一個基礎框架。我們將端到端策略與機器人作業系統 (ROS) 中最先進的 Nav2 導航堆疊進行基準測試。我們還透過展示在 Isaac 模擬器中訓練的策略對真實世界機器人的零樣本轉移能力，涵蓋了虛實轉移過程。不同模擬機器人的測試進一步證明了這一點，這些測試顯示了學習策略的泛化能力。最後，基準測試證明了與 Nav2 相當的性能，為在客製化機器人平台上快速部署最先進的端到端局部路徑規劃器打開了大門，但更重要的是，透過擴展狀態和動作空間或任務定義以實現更複雜的任務，進一步擴展了可能性。總體而言，本文介紹了在使用 Isaac Sim 訓練、Gazebo 測試和 ROS 2 進行真實機器人即時推斷的情況下，部署用於局部路徑規劃和避障的 RL 策略時最重要的步驟和需要考慮的方面。程式碼可在 <https://github.com/sahars93/RL-Navigation> 取得。

Index Terms-Reinforcement learning (RL); Deep reinforcement learning; Sim-to-real transfer; Mobile robotics; Local planning; Obstacle avoidance; Gazebo; ROS 2; Nav2; End-to-end control.

索引詞彙—強化學習 (RL)；深度強化學習；模擬到現實遷移；移動機器人；局部路徑規劃；避障；Gazebo；ROS 2；Nav2；端到端控制。

1 Introduction

1 緒論

Reinforcement learning (RL) stands at the forefront of enabling complex control and facilitating advanced behaviors in various types of robots. This advancement holds promise for revolutionizing robotics, empowering machines to interact with their environments. RL algorithms are widely used across classic tasks including locomotion, navigation, or manipulation, among others. Indeed, recent years have seen unprecedented improvements in the ability of quadruped robots [1], wheeled-legged robots [2], drone racing [3], humanoids [4],

強化學習 (RL) 站在實現複雜控制和促進各類機器人進階行為的最前沿。這項進展有望徹底改變機器人技術，使機器能夠與其環境互動。RL 演算法廣泛應用於經典任務，包括移動、導航或操作等。事實上，近年來，四足機器人 [1]、輪足機器人 [2]、無人機競速 [3]、人形機器人 [4] 的能力取得了前所未有的進步，

S. Salimpour, J. Heikkonen, and T. Westerlund were with the University of Turku, Finland.

S. Salimpour、J. Heikkonen 和 T. Westerlund 曾任職於芬蘭圖爾庫大學。

Emails: {sahars, jukhei, toveve}@utu.fi

電子郵件：{sahars, jukhei, toveve}@utu.fi

J. Peña-Queralta and D. Paez-Granados were with the Swiss Federal School of Technology in Zurich - ETH Zurich, 8092 Switzerland.

J. Peña-Queralta 和 D. Paez-Granados 曾任職於瑞士蘇黎世聯邦理工學院 (ETH Zurich)，郵遞區號 8092，瑞士。

Emails: {jorge.penaqueralta, diego.paez}@hest.ethz.ch.

電子郵件：{jorge.penaqueralta, diego.paez}@hest.ethz.ch。

or bipedal robot sports [2]. Also, in the automation of machinery such as hydraulic excavators [5].

或雙足機器人運動 [2]。此外，也應用於液壓挖掘機等機械的自動化 [5]。

In the majority of these problems, a policy is trained to map a control input, together with the robot sensory inputs, to joint-level actuation. For example, [1] maps from body-state velocities to steps, while [4] focuses on whole-body motion planning. Throughout this large variety of use cases and robotic systems, often focused on dexterous manipulation, or motion planning with a large number of degrees of freedom and/or uncertainty [6]-[8], the field has established a range of commonly used approaches and, importantly, simulation tools. The latter include NVIDIA Isaac Sim or Orbit [9], [10], MuJoCo [2], [4], or Flightmare [3], among others.

在大多數這些問題中，策略被訓練來將控制輸入與機器人感測器輸入一起映射到關節級致動。例如，[1] 將身體狀態速度映射到步態，而 [4] 則專注於全身運動規劃。在如此多樣化的用例和機器人系統中，通常專注於靈巧操作，或具有大量自由度和/或不確定性的運動規劃 [6]-[8]，該領域已經建立了一系列常用的方法，以及重要的模擬工具。後者包括 NVIDIA Isaac Sim 或 Orbit [9]、[10]、MuJoCo [2]、[4] 或 Flightmare [3] 等。

Beyond low-level control and motion planning from proprioceptive sensory inputs, RL has also been studied within the more general perspective of mobile robotics for local or global navigation, and high-

除了來自本體感覺輸入的低階控制和運動規劃之外，RL 也已在行動機器人領域的更廣泛視角中進行了研究，用於局部或全域導航，以及高階規劃和自主性。

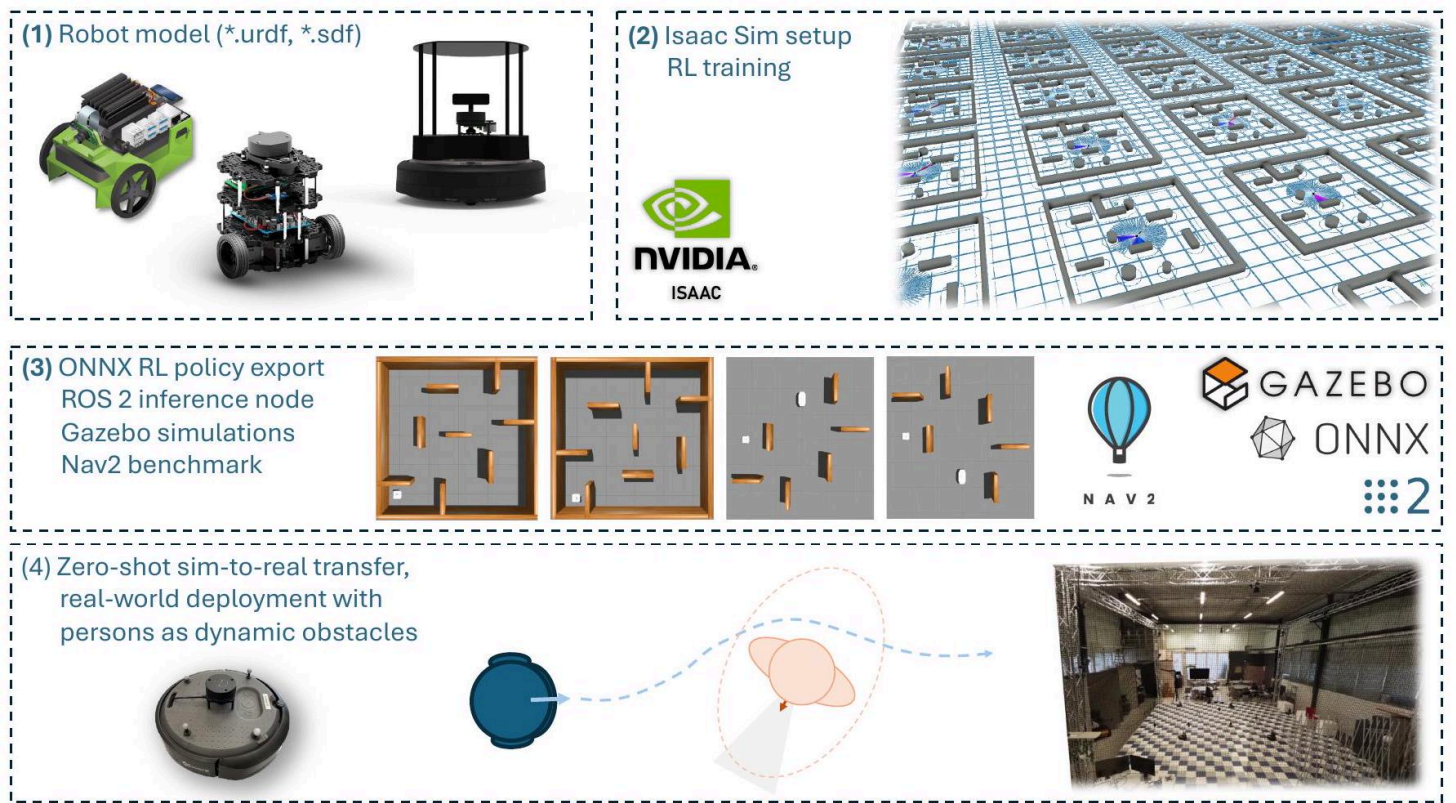


Figure 1: Fig. 1: Conceptual illustration of the sim-to-real workflow described in this article. In the first step, we utilize different existing robot models, while also describing the Isaac model importer functionality in Section 3. In the second step, we describe key considerations in terms of RL policy training in Section 4, and the setup of different static and dynamic environments. In the third step, we provide template Robot Operating System (ROS 2) nodes, and guidance on Gazebo testing in Sections 4 and 5. Additionally, we benchmark the performance to the state-of-the-art Nav2 navigation and planning algorithms. Finally, in the fourth step, we also demonstrate the zero-shot sim-to-real transfer capabilities in Section 5.

圖 1：圖 1：本文所述的模擬到現實工作流程的概念圖。在第一步中，我們利用不同的現有機器人模型，同時也在第 3 節中描述了 Isaac 模型導入器功能。在第二步中，我們在第 4 節中描述了 RL 策略訓練的關鍵考量，以及不同靜態和動態環境的設置。在第三步中，我們提供了模板機器人作業系統 (ROS 2) 節點，並在第 4 節和第 5 節中提供了 Gazebo 測試指南。此外，我們還將性能與最先進的 Nav2 導航和規劃演算法進行了基準測試。最後，在第四步中，我們還在第 5 節中展示了零樣本模擬到現實的遷移能力。

level planning and autonomy. This applies to both complex robots such as quadrupeds [1], to wheeled robots there motion planning is more straightforward. The level of standardization and the depth of the study of sim-to-real transfer for navigation task, however, is shallower [11]. Through this article, we aim to give more insight into such an a priori more rudimentary and classical problem, but where RL controllers can also play an important role as the field solidifies. Our focus is on providing an step-by-step approach to training RL policies for mobile robot navigation from scratch, and describing the transition from simulation to reality (see Figure 1).

高階規劃和自主性。這適用於複雜的機器人，例如四足機器人 [1]，也適用於運動規劃更直接的輪式機器人。然而，導航任務的標準化程度和模擬到現實遷移研究的深度較淺 [11]。透過本文，我們旨在對這種先驗上更基本和經典的問題提供更多見解，但隨著該領域的鞏固，RL 控制器也能發揮重要作用。我們的重點是提供一個從頭開始訓練行動機器人導航 RL 策略的逐步方法，並描述從模擬到現實的過渡（參見圖 1）。

From this point forward, we constraint use cases to path planning and local navigation for mobile robots from an initial position to a target destination. The sensory focus is exteroceptive, aiming at training end-to-end policies that enable navigation without collisions in both simulated and realworld scenarios. This area, including complex static and dynamic environments, has been the subject of extensive study in recent literature [12]. Deep

從此時起，我們將使用案例限制在移動機器人從初始位置到目標目的地的路徑規劃和局部導航。感官重點是外感受性的，旨在訓練端到端策略，使其能夠在模擬和現實世界場景中無碰撞地導航。這個領域，包括複雜的靜態和動態環境，一直是近期文獻中廣泛研究的主題 [12]。深度

reinforcement learning algorithms have emerged as a promising solution to this challenge [13]. Such RL algorithms have

shown significant potential in navigation tasks for different types of robots and sensors such as LiDAR, RGB camera, and RGB-D camera [14].

強化學習演算法已成為解決此挑戰的一個有前景的方案 [13]。此類強化學習演算法在不同類型機器人（如 LiDAR、RGB 攝影機和 RGB-D 攝影機）的導航任務中展現出巨大的潛力 [14]。

Some representative examples are the following. [15] proposes the usage of the Advantage ActorCritic algorithm to navigate their robot in an environment with 3D obstacle avoidance, achieved through the fusion of a 2D laser scanner with an RGB-D camera in a self-implemented simulator. In [13], a soft actor-critic algorithm has been utilized to train and test an obstacle avoidance model for a differential drive robot. Similar to most lidarbased navigation studies, factors such as relative distance to the target point, lidar scan data, and the robot's speed are employed to determine the velocity necessary to drive toward the target point. Many of these studies have conducted their training processes within the Gazebo simulation environment [16], [17]. In [18], a model-free, on-policy

一些具代表性的例子如下。[15] 提出使用 Advantage Actor-Critic 演算法，透過在自行實作的模擬器中融合 2D 雷射掃描儀與 RGB-D 攝影機，使其機器人在具有 3D 避障的環境中導航。在 [13] 中，軟演員-評論家演算法被用於訓練和測試差速驅動機器人的避障模型。與大多數基於光達的導航研究類似，諸如與目標點的相對距離、光達掃描數據和機器人速度等因素被用來確定驅動至目標點所需的速度。這些研究中有許多是在 Gazebo 模擬環境中進行訓練的 [16]、[17]。在 [18] 中，採用無模型、在線的

deep RL approach is employed to train the control policy within TensorFlow agents simulation. It aims to guide the drone at high speeds through gates while observing the current robot state's estimate, the gate's relative pose using an onboard camera, and the previous action.

深度強化學習方法，在 TensorFlow agents 模擬中訓練控制策略。其目標是引導無人機以高速穿過閘門，同時觀察當前機器人狀態的估計、使用機載攝影機的閘門相對姿態以及先前的動作。

A significant portion of existing studies, illustrated by the previous examples, are confined to simulations. Additionally, approaches are often tailored or the papers concentrate on either particular robots or specific considerations of a use-case. Furthermore, the lack of standardized benchmarks and open-source implementations hinders validation and comparison [19]. In this article, we address these gaps by providing detailed implementation steps, demonstrating the process of training an RL agent from simulation to real-world deployment on a generic wheeled robot. Specifically, through this magazine article, we also aim to introduce a broader audience to utilizing the state-of-the-art NVIDIA Omniverse Isaac Sim to achieve autonomous local planning and obstacle avoidance from the ground up. We delve into the key challenges and compare different approaches to training. Through learning from robot's interactions with the environment—specifically by evaluating the presence and proximity of nearby walls and obstacles—these algorithms enable robots to make informed decisions and adapt to new situations, thereby improving their navigation capabilities in unknown environments. This makes reinforcement learning a potentially compelling approach for advancing mobile robot navigation.

現有研究的很大一部分（如前述範例所示）僅限於模擬。此外，方法通常是為特定機器人或特定用例考量而量身定制，或者論文只專注於其中之一。此外，缺乏標準化的基準和開源實作阻礙了驗證和比較 [19]。在本文中，我們透過提供詳細的實作步驟，展示了在通用輪式機器人上從模擬到實際部署訓練 RL 代理的過程，從而解決了這些空白。具體來說，透過這篇雜誌文章，我們還旨在向更廣泛的受眾介紹如何利用最先進的 NVIDIA Omniverse Isaac Sim，從頭開始實現自主局部規劃和避障。我們深入探討了關鍵挑戰，並比較了不同的訓練方法。透過從機器人與環境的互動中學習——特別是透過評估附近牆壁和障礙物的存在和接近程度——這些演算法使機器人能夠做出明智的決策並適應新情況，從而提高它們在未知環境中的導航能力。這使得強化學習成為推進行動機器人導航的潛在引人注目的方法。

The main contributions of this article with respect to the available literature are the following. First, we provide an in-depth description of the state-of-the-art Isaac Sim simulator for RL-based navigation of wheeled robots. Second, we discuss in detail different training strategies (e.g., curriculum learning) and the key aspects to account for when defining tasks, including reward function design. Finally, we demonstrate sim-to-real transfer of end-to-end RL policies across robots, while benchmarking to classical state-of-the-art navigation approaches. Overall, we aim to provide a more comprehensive analysis of the problem of training RL-based local planners for navigation than existing literature, making learning-based approaches to ground robot navigation accessible to a larger audience. Throughout the article, we assume familiarity with

basic RL concepts.

本文相對於現有文獻的主要貢獻如下。首先，我們深入描述了用於輪式機器人基於 RL 導航的最先進的 Isaac Sim 模擬器。其次，我們詳細討論了不同的訓練策略（例如課程學習）以及在定義任務時需要考慮的關鍵方面，包括獎勵函數設計。最後，我們展示了端到端 RL 策略在機器人之間的模擬到實際遷移，同時與經典的最先進導航方法進行了基準測試。總體而言，我們旨在比現有文獻更全面地分析訓練基於 RL 的局部規劃器以進行導航的問題，使基於學習的地面機器人導航方法能夠被更廣泛的受眾所接受。在整篇文章中，我們假設讀者熟悉基本的 RL 概念。

The rest of the manuscript is organized as follows. Section 2 covers the use of Isaac Sim as a simu-

手稿的其餘部分組織如下。第 2 節介紹了 Isaac Sim 作為模擬器在 RL 策略訓練中的應用。

lator for RL policy training. Section 3 then describes in more detail the different elements required to set up a training workflow. In Section 4, we delve into specific fine-tuning aspects of the proximal policy optimization (PPO) algorithm, one of the de-facto standards in the field, including reward modeling and model and training hyperparameters. Simulation and experimental results, with a focus on describing a sim-to-real transfer based on ROS 2 and the Gazebo simulator, are introduced in Section 5. Section 6 discusses and concludes the work.

第 3 節更詳細地描述了設定訓練工作流程所需的各種元素。在第 4 節中，我們深入探討了近端策略優化 (PPO) 演算法的特定微調方面，PPO 是該領域事實上的標準之一，包括獎勵建模以及模型和訓練超參數。第 5 節介紹了模擬和實驗結果，重點描述了基於 ROS 2 和 Gazebo 模擬器的模擬到實際遷移。第 6 節討論並總結了這項工作。

2 RL With Isaac Sim

2 搭配 Isaac Sim 的強化學習

The introduction of the NVIDIA Omniverse Isaac Gym and Orbit frameworks have arguably aided in widening of audience and applications of deep reinforcement learning research.

NVIDIA Omniverse Isaac Gym 和 Orbit 框架的推出，無疑有助於擴大深度強化學習研究的受眾和應用。

2.1 Isaac Sim

Isaac Sim, a GPU-based general-purpose physics simulation platform from Nvidia, serves as an extensible robotics simulator that empowers designers, researchers, and developers to create, test, and train AI-based robots such as wheeled robots, legged robots, and drones. Leveraging the power of NVIDIA Omniverse, Isaac Sim provides scalable, photorealistic, and physically accurate virtual environments for high-fidelity simulations. It can simulate realistic sensor models such as camera, lidar, and IMU, and a variety of objects and scenes, enabling tasks such as manipulation, navigation, synthetic data generation, and various computer vision applications through Python, ROS integration, and Isaac SDK.

Isaac Sim 是 Nvidia 推出的一個基於 GPU 的通用物理模擬平台，它是一個可擴展的機器人模擬器，讓設計師、研究人員和開發人員能夠建立、測試和訓練基於 AI 的機器人，例如輪式機器人、足式機器人和無人機。Isaac Sim 利用 NVIDIA Omniverse 的強大功能，提供可擴展、逼真且物理精確的虛擬環境，以進行高擬真模擬。它能模擬逼真的感測器模型，例如攝影機、光達和慣性測量單元 (IMU)，以及各種物件和場景，透過 Python、ROS 整合和 Isaac SDK，實現操作、導航、合成資料生成和各種電腦視覺應用等任務。

2.2 RL in Isaac Sim / Gym

2.2 Isaac Sim / Gym 中的強化學習

Omniverse Isaac Gym is an extension for reinforcement learning in robotics which is built on top of NVIDIA Isaac Sim. Isaac Gym is highly parallelized simulations by conducting both physics simulation and policy training on the GPU through an API, based on the vectorization of observations and actions. This framework offers a straightforward interface for training RL agents and supports various RL algorithms. In the latest releases of Isaac Gym, RL Games is introduced as the default library for running example environments. Whether it is training robotic agents to perform complex tasks, fine-tuning and optimizing RL policies, or evaluating their performance, Isaac Gym provides a bridge between the simulation

environment and RL algorithms.

Omniverse Isaac Gym 是 NVIDIA Isaac Sim 之上建構的機器人強化學習擴充功能。Isaac Gym 透過基於觀察和動作向量化的 API，在 GPU 上進行物理模擬和策略訓練，實現高度平行化的模擬。此框架為訓練強化學習代理提供了直接的介面，並支援各種強化學習演算法。在 Isaac Gym 的最新版本中，RL Games 被引入作為執行範例環境的預設函式庫。無論是訓練機器人代理執行複雜任務、微調和最佳化強化學習策略，還是評估其效能，Isaac Gym 都提供了模擬環境和強化學習演算法之間的橋樑。

More recent RL framework, also powered by NVIDIA Isaac Sim, are Orbit and Isaac Lab. Orbit offers a comprehensive suite of features, including support for various robot platforms, sensors, teleoperation, imitation Learning, and motion planning across diverse robotic applications, while Isaac Lab provides more comprehensive environments for different types of robots and tasks ¹. In these new frameworks, beginners may encounter a steeper learning curve. This article, therefore, focuses on the more documented and tested Isaac Gym as the first tutorial for straightforward path planning and navigation implementation, which can be further developed in Isaac Lab for more advanced features.

更新的強化學習框架，同樣由 NVIDIA Isaac Sim 提供支援，是 Orbit 和 Isaac Lab。Orbit 提供了一套全面的功能，包括支援各種機器人平台、感測器、遙操作、模仿學習以及跨多種機器人應用的運動規劃，而 Isaac Lab 則為不同類型的機器人和任務提供了更全面的環境 ¹。在這些新框架中，初學者可能會遇到更陡峭的學習曲線。因此，本文將重點放在文檔更完善且經過測試的 Isaac Gym，作為直接路徑規劃和導航實施的第一個教學，這可以在 Isaac Lab 中進一步開發以實現更進階的功能。

3 RL Workflow

3 強化學習工作流程

In Isaac Gym, the development and simulation of a customized RL agent navigation task needs a few basic steps for effective simulation, training, and testing. This includes defining the simulation environment and robot, crafting a Python script for the task class to specify goals, reward computation, and reset management, and utilizing two YAML configuration files—one for task parameters and another for training parameters—to complete the task. This section provides an overview of these essential core components.

在 Isaac Gym 中，客製化 RL 代理導航任務的開發和模擬需要一些基本步驟，以實現有效的模擬、訓練和測試。這包括定義模擬環境和機器人、編寫用於任務類別的 Python 腳本以指定目標、獎勵計算和重置管理，以及利用兩個 YAML 配置檔案（一個用於任務參數，另一個用於訓練參數）來完成任務。本節概述了這些基本核心組件。

3.1 Robot And Environments

3.1 機器人與環境

Isaac Gym offers a user-friendly API for creating and configuring scenes with custom robots and objects. Many ROS users utilize the Unified Robot Description Format (URDF), a popular format for describing the basic robot cell and geometry, and practical applications. Isaac Sim supports various file formats, including URDF, Multi-Joint dynamics with Contact (MJCF), and Universal Scene Description (USD). It is feasible to incorporate custom robots from URDF and MJCF files into tasks, or alternatively, convert them to USD format using the Isaac Sim Importer extensions. Two robot models were employed in our mobile robot navigation task, the modified Isaac built-in Jetbot robot with Lidar sensor (Figure 2a) and the USD model of the Turtlebot3-Waffle from the TurtleBot3 Simulation ROS Package, converted through the Isaac URDF Importer as shown in Figure. 2b for additional experiments. To correctly import the mobile robot from the URDF file, the

“Fixed Base Link” must be unchecked, and the “Joint Drive Type” must be set

Isaac Gym 提供了一個使用者友善的 API，用於創建和配置包含自訂機器人和物件的場景。許多 ROS 使用者利用統一機器人描述格式 (URDF)，這是一種描述基本機器人單元和幾何形狀以及實際應用的流行格式。Isaac Sim 支援各種檔案格式，包括 URDF、多關節動力學與接觸 (MJCF) 和通用場景描述 (USD)。將 URDF 和 MJCF 檔案中的自訂機器人整合到任務中是可行的，或者，也可以使用 Isaac Sim 匯入器擴充功能將它們轉換為 USD 格式。在我們的行動機器人導航任務中使用了兩種機器人模型：修改後的 Isaac 內建 Jetbot 機器人（帶有 Lidar 感測器）（圖 2a）和來自 TurtleBot3 模擬 ROS 套件的 Turtlebot3-Waffle 的 USD 模型，透過 Isaac URDF 匯入器轉換，如圖 2b 所示，用於額外實驗。為了正確從 URDF 檔案匯入行動機器人，「固定基座連結」必須取消勾選，「關節驅動類型」必須設定為

to “Velocity”. When working on specific tasks, one can create, modify, and save custom scenes within Isaac Sim and add them as USD files into the task as shown in Figure. 2c.

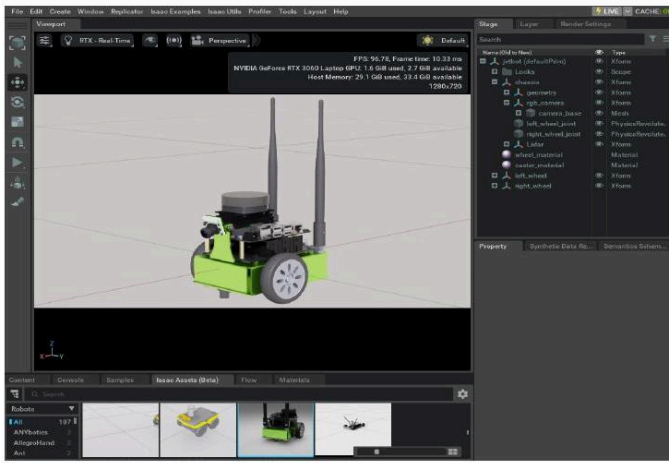
「速度」。在執行特定任務時，可以在 Isaac Sim 中建立、修改和儲存自訂場景，並將其作為 USD 檔案新增到任務中，如圖 2c 所示。

3.2 Task

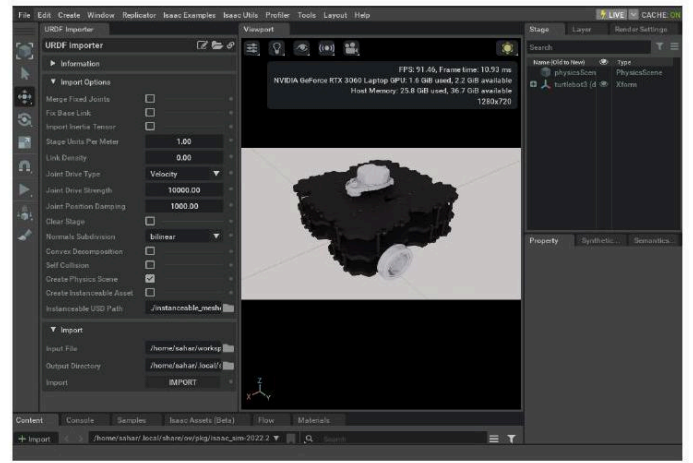
3.2 任務

A variety of reinforcement learning tasks are provided at Omniverse Isaac Gym extension, where main functionalities such as performing episode resets, applying actions, collecting observations, and computing rewards are implemented in this task class. Our JetbotTask, inherits from the BaseTask class in omni.isaac.core, comprises several key components. The general structure of the definition of each component of a new RL task is shown in Listing 1 The Initialization phase, the init function, sets initial configurations for the environment. The initial setup for each task is detailed in its dedicated task YAML file. An overview of this file, along with a few sample parameters, is presented in 3.2 One can specify parameters related to the environment and simulation within this file. These include the number of environments, various sensors and USD configurations for the robot and objects, specifying CPU or GPU pipeline, and applying noise to observations, actions, and other properties in the domain randomization part. Action and observation space and other parameters such as the episode’s length are also defined in the init function. The set_up_scene function is for setting up the scene by creating ArticulationView or RigidPrimView objects. This function involves defining the scene, sensor, and robot, or loading assets from USD, URDF, and MJCF file formats. The get_observation function generates the observation space using Lidar ranges, information on the target’s relative position, and the robot’s state. Computations required before stepping into the physics simulation, such as applying actions to move the robot based on policy decisions or resetting the environment, occur in the pre_physics_step function. The calculation of rewards, resets, and extra buffers is handled in the calculate_metrics function. Finally, determining which environments need resetting is done in the last function of the task. Besides the task config file, each task is accompanied by its configuration file containing training parameters such as the model and network structures, and the PPO parameters such as the learning rate, as shown in 3. These parameters are passed through rlgames_train.py.

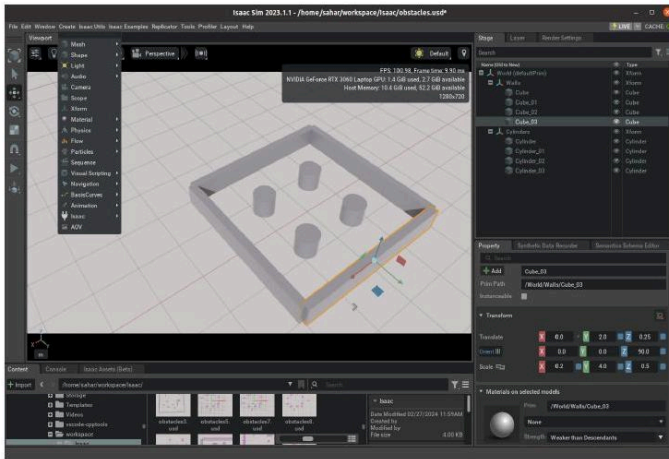
Omniverse Isaac Gym 擴充功能提供了多種強化學習任務，其中主要功能如執行回合重置、應用動作、收集觀察結果和計算獎勵等都實作在此任務類別中。我們的 JetbotTask 繼承自 omni.isaac.core 中的 BaseTask 類別，包含幾個關鍵組件。新 RL 任務每個組件定義的一般結構如清單 1 所示。初始化階段，即 init 函數，設定環境的初始配置。每個任務的初始設定在其專用的任務 YAML 檔案中詳細說明。此檔案的概述以及一些範例參數如 3.2 所示。可以在此檔案中指定與環境和模擬相關的參數。這些參數包括環境數量、機器人和物件的各種感測器和 USD 配置、指定 CPU 或 GPU 管線，以及在領域隨機化部分對觀察結果、動作和其他屬性應用雜訊。動作和觀察空間以及其他參數（例如回合長度）也在 init 函數中定義。set_up_scene 函數用於透過建立 ArticulationView 或 RigidPrimView 物件來設定場景。此函數涉及定義場景、感測器和機器人，或從 USD、URDF 和 MJCF 檔案格式載入資產。get_observation 函數使用光達範圍、目標相對位置資訊和機器人狀態來產生觀察空間。在進入物理模擬之前所需的計算，例如根據策略決策應用動作來移動機器人或重置環境，發生在 pre_physics_step 函數中。獎勵、重置和額外緩衝區的計算在 calculate_metrics 函數中處理。最後，在任務的最後一個函數中確定哪些環境需要重置。除了任務配置檔案，每個任務都附帶其配置檔案，其中包含訓練參數，例如模型和網路結構，以及 PPO 參數，例如學習率，如 3 所示。這些參數透過 rlgames_train.py 傳遞。



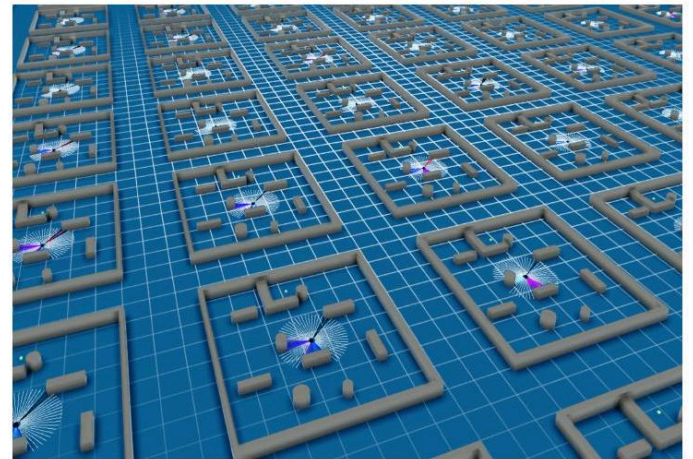
(a) Isaac Jetbot robot



(b) URDF Importer Extension



(c) A custom environment in Isaac Sim



(d) Isaac Gym

Figure 2: Fig. 2: Robots and Environments

圖 2：圖 2：機器人與環境

```
class NewTask (RLTask):
    def __init__(
        config # Defined in the task.yaml
    )
    def set_up_scene(self, scene):
        self.get_robot()
        self._robots = RobotView()
        scene.add(self._robots)
        return
    def get_robot():
        robot = Robot()
        add_reference_to_stage()
    def get_observations(self):
        self.obs_buf[:] = obs
        return observations
    def pre_physics_step(self, actions):
        self.reset_idx(reset_env_ids)
```



```

        actions = actions.to(self._device)
def calculate_metrics(self):
    ...
def is_done(self):
    ...

```

Listing 1: General structure of a new RL task definition in OmniIsaacGym.

清單 1：OmniIsaacGym 中新 RL 任務定義的一般結構。

3.3 From Isaac To Gazebo

3.3 從 Isaac 到 Gazebo

Before deploying navigation policies in the real world, we evaluate the trained policy within the

在實際部署導航策略之前，我們會在

```

name: Robot
env:
  numEnvs:
sim:
  use_gpu_pipeline: ...
  use_flatcache: ...
  PhysX:
    use_gpu: ...
  Robot:
    ...
  object:
    ...
domain_randomization:
  observations:
    ...
  actions:
    ...

```



Listing 2: Core components of task.yaml.

清單 2：task.yaml 的核心元件。

Gazebo simulation environment. Gazebo, a widely adopted simulator in the global robotics community, directly interfaces with the ROS through userfriendly packages. This integration facilitates the creation of accurate simulations, and the outcomes obtained can be directly implemented on the real robots with only ROS installed, regardless of their software

architecture. There are many Gazebo simulation packages available, featuring different dif-

Gazebo 模擬環境中評估訓練好的策略。Gazebo 是全球機器人社群廣泛採用的模擬器，透過使用者友善的套件直接與 ROS 介面。這種整合有助於建立精確的模擬，並且無論其軟體架構如何，所獲得的結果都可以直接在僅安裝 ROS 的真實機器人上實施。有許多 Gazebo 模擬套件可用，具有不同的差異。

```
params:
  algo:
    name: ...
  model:
    name: ...
  network:
    mlp:
      units: ...
      ...
    rnn:
      name: ...
      ...
  config:
    learning_rate: ...
    minibatch_size: ...
    mini_epochs: ...
    critic_coef: ...
```



Listing 3: Core components of train.yaml.

清單 3：train.yaml 的核心元件。

ferential drive robots equipped with Lidar sensors.

差動驅動機器人配備光達感測器。

In order to test in the Gazebo simulator, the trained model weights are exported in the format of ONNX model and integrated into a ROS node. The ONNX model can be imported and utilized for both simulation and real-world scenarios.

為了在 Gazebo 模擬器中進行測試，訓練好的模型權重會以 ONNX 模型的格式匯出，並整合到 ROS 節點中。ONNX 模型可以匯入並用於模擬和實際場景。

3.4 ROS 2 Node Deployment

3.4 ROS 2 節點部署

A ROS2 node is defined to handle the ROS operations within the robots. Upon initialization, such as the target's position, it sets up subscriptions to topics such as the Scan topic to get the Lidar ranges and the robot's position through the Odometry topic, along with a publisher for cmd_vel commands. Additionally, the class integrates an ONNX model, for inference. Various callback functions are implemented, such as Odometry_callback(), and scan_callback(), to process incoming messages from subscribed topics. Finally, the send_control() function generates robot control commands based on the

model's outputs and publishes them via cmd_vel.

定義了一個 ROS2 節點來處理機器人內部的 ROS 操作。在初始化時，例如目標位置，它會設定訂閱主題，例如 Scan 主題以獲取 Lidar 範圍，以及透過 Odometry 主題獲取機器人位置，同時還有一個用於 cmd_vel 命令的發布器。此外，該類別整合了一個 ONNX 模型用於推論。實作了各種回呼函數，例如 Odometry_callback() 和 scan_callback()，以處理來自訂閱主題的傳入訊息。最後，send_control() 函數根據模型的輸出生成機器人控制命令，並透過 cmd_vel 發布它們。

4 RL for Navigation

4 導航的強化學習

This section covers key aspects that need to be customized based on the robot, available sensors, or the overall use-case or general optimization objectives. These include the definition of observation and action spaces, reward modeling, and hyperparameter settings for both the model and the training and environment setup.

本節涵蓋了需要根據機器人、可用感測器、整體使用案例或一般最佳化目標進行客製化的關鍵面向。這些面向包括觀察和動作空間的定義、獎勵建模，以及模型、訓練和環境設定的超參數設定。

4.1 Observation State

4.1 觀察狀態

The primary objective of an RL policy is to optimize the cumulative reward, by effectively navigating the interactions between the agent and its environment.

RL 策略的主要目標是透過有效引導代理程式與其環境之間的互動，來最佳化累積獎勵。

```
class ROSNode(Node):
    def __init__(self):
        super().__init__('ROS_node')
        self.Pose_subscription = ...
        self.LidarRages_subscription = ...
        self.cmd_vel_publisher = ...
        self.ort_model =
            ort.InferenceSession("model.onnx")
    def Odometry_callback(self, msg):
        self.robot_position = ...
        self.robot_orientation = ...
    def scan_callback(self, msg):
        self.ranges = ...
    def send_control(self):
        observation = ...
        outputs = self.ort_model.run()
        self.cmd_vel_publisher.publish(twist)

def main():
    if __name__ == '__main__':
```



Listing 4: ROS2 Node for real-time inference.

列表 4：用於即時推論的 ROS2 節點。

In our navigation task, at each time step, the observation state o_t comprises 2D lidar scans with 6degree resolution (120 scans) in the range of 0.15, 3 shown as L_t , relative goal position as a 2D vector representing the robot's relative distance, d_t , and angle, θ_t , to the goal in polar coordinates, and the linear velocity v_{t-1} from the previous time step, along with the angular velocity ω_{t-1} . In (1), a_t is the 2D action space, defined as linear velocity v_t and angular velocity ω_t , and the DRL policy, π , maps the observed state o_t , into the updated action, as the linear velocity in the range of $[0.1, 0.5](\text{m/s})$ and angular velocity in the range of $[-0.5, 0.5]$, to direct the robot towards its goal while avoiding collisions.

在我們的導航任務中，每個時間步的觀察狀態 o_t 包含 2D 光達掃描，其解析度為 6 度（120 次掃描），範圍介於 0.15 和 3 之間，如 L_t 所示；相對目標位置，表示為機器人相對於目標的 2D 向量， d_t ，以及極座標中的角度， θ_t ；以及前一個時間步的線速度 v_{t-1} 和角速度 ω_{t-1} 。在 (1) 中， a_t 是 2D 動作空間，定義為線速度 v_t 和角速度 ω_t ，而 DRL 策略 π 將觀察到的狀態 o_t 映射到更新後的動作，即線速度範圍介於 $[0.1, 0.5](\text{m/s})$ 和角速度範圍介於 -0.5 到 0.5 之間，以引導機器人朝向目標同時避免碰撞。

$$\begin{aligned} a_t &: v_t, \omega_t \\ o_t &: L_t, d_t, \theta_t, v_{t-1}, \omega_{t-1} \\ a_t &\sim \pi(o_t | a_t) \end{aligned}$$

4.2 Rewards

4.2 獎勵

In the reinforcement learning framework, the reward function plays a crucial role in assessing the effectiveness of the robot's actions. In navigation tasks, agents receive rewards based on goal achievement, collision avoidance, and time. Sparse rewards can hinder convergence, so the reward structure can be adjusted to better suit the task and enhance learning efficiency. In our navigation task, the primary aim is to ensure the robot avoids collisions and reaches its destination swiftly. The reward function, denoted as R_t in (2), is designed with three main components. The robot receives rewards as

在強化學習框架中，獎勵函數對於評估機器人動作的有效性扮演著關鍵角色。在導航任務中，代理人根據目標達成、避碰和時間獲得獎勵。稀疏獎勵可能會阻礙收斂，因此可以調整獎勵結構以更好地適應任務並提高學習效率。在我們的導航任務中，主要目標是確保機器人避開碰撞並迅速到達目的地。獎勵函數，在 (2) 中表示為 R_t ，由三個主要部分設計。機器人獲得獎勵，因為

it progressively reduces its distance from the target, defined as r_{distance} , and to avoid the collision, an exponential penalty, $r_{\text{collision}}$, is applied to the robot as it gets closer to the obstacles, increasing as it approaches the threshold Min, as the closest possible distance. Furthermore, to determine the shortest path, the robot is rewarded based on the time it takes to reach the target. This means that after reaching the target, the reward is calculated based on the remaining episode length.

它逐漸減少與目標的距離，定義為 r_{distance} ，為了避免碰撞，當機器人接近障礙物時，會施加指數懲罰 $r_{\text{collision}}$ ，隨著它接近閾值 Min（作為最近的可能距離）而增加。此外，為了確定最短路徑，機器人會根據到達目標所需的時間獲得獎勵。這意味著在到達目標後，獎勵是根據剩餘的劇集長度計算的。

$$R_t = \begin{cases} r_{\text{distance}} & : + (d_{t-1} - d_t) \\ r_{\text{collision}} & : - (e^{-\min_{\text{range}}}) \quad \min_{\text{range}} < \text{Min} \\ r_{\text{time}} & : + (\text{remaining steps}) \\ \text{Goal} & : + r_1 \quad \text{Reset: True} \\ \text{Collision} & : - r_2 \quad \text{Reset: True} \\ \text{Max length} & : - r_3 \quad \text{Reset: True} \end{cases}$$

In the reward function, we introduce additional conditions that signal the end of an episode. In addition to the previously discussed rewards, we define three fixed rewards that serve as reset flags. The first is awarded upon reaching the target, defined as Goal, Collision is a fixed negative reward for collisions when the robot gets too close to obstacles beyond the Min threshold, and Max length is a penalty for surpassing the maximum episode length before reaching to the target. These three conditions collectively signify the conclusion of an episode in the `is_done` function in the Listing 1. The reward

amounts can be defined based on the size and type of the robot, avoidance distance, and the environment. In our Jetbot navigation task, the Min is 25 cm and all fixed rewards r_1 , r_2 , and r_3 are set to 30 .

在獎勵函數中，我們引入了額外的條件來表示劇集的結束。除了前面討論的獎勵之外，我們定義了三個固定的獎勵作為重置標誌。第一個是在達到目標時授予的，定義為 Goal，Collision 是當機器人過於接近超出 Min 閾值的障礙物時的固定負獎勵，而 Max length 是在達到目標之前超過最大劇集長度的懲罰。這三個條件共同表示 Listing 1 中 is_done 函數中劇集的結束。獎勵金額可以根據機器人的大小和類型、避讓距離和環境來定義。在我們的 Jetbot 導航任務中，Min 為 25 公分，所有固定獎勵 r_1 , r_2 和 r_3 都設定為 30 。

In the literature, the three main components—goal, collision, and time—are essential, with their weights varying. A large penalty for collisions may limit exploration, while a high reward for the target might lead to overfitting to a specific target point, reducing generalization. Typically, training begins with equal weights, and adjustments are made based on the specific task.

在文獻中，目標、碰撞和時間這三個主要組成部分至關重要，其權重各不相同。對碰撞施加過大的懲罰可能會限制探索，而對目標給予過高的獎勵可能會導致對特定目標點的過度擬合，從而降低泛化能力。通常，訓練會從相等的權重開始，並根據具體任務進行調整。

4.3 Model Definition

4.3 模型定義

We set the episode length to a maximum of 1200 steps, with 64 environments running for 1500 iterations. Our implementation revolves around the Actor-Critic algorithm designed for continuous spaces. The actor-critic algorithm is a reinforcement learning technique that merges policy-driven (Actor) and value-driven (Critic) approaches. The Actor

我們將每個回合的長度設定為最多 1200 步，並讓 64 個環境運行 1500 次迭代。我們的實作圍繞著為連續空間設計的 Actor-Critic 演算法。Actor-Critic 演算法是一種強化學習技術，它結合了策略驅動（Actor）和價值驅動（Critic）的方法。Actor

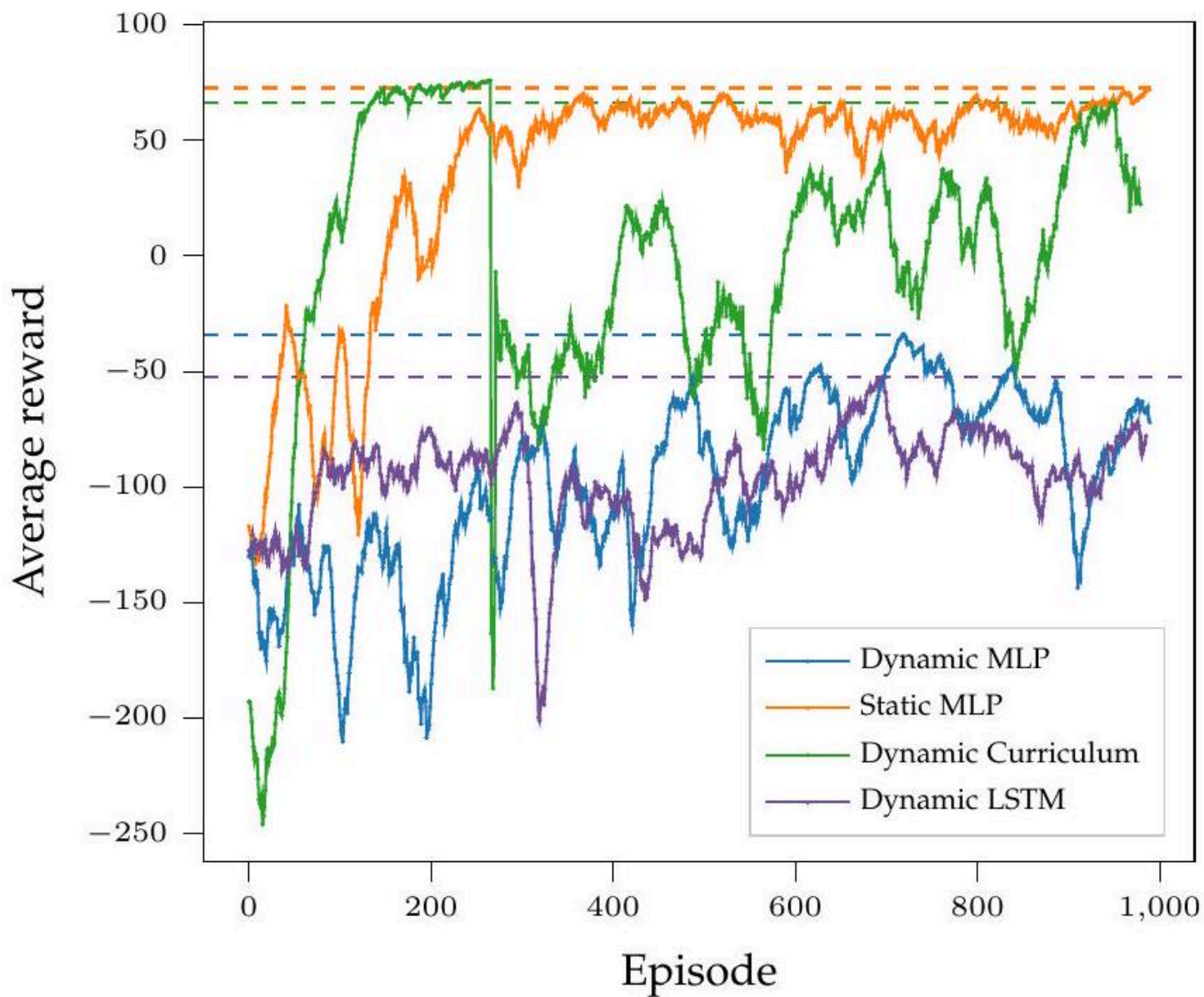


Figure 3: Fig. 3: Episodic returns during training using Isaac Sim

圖 3：圖 3：使用 Isaac Sim 訓練期間的回合報酬

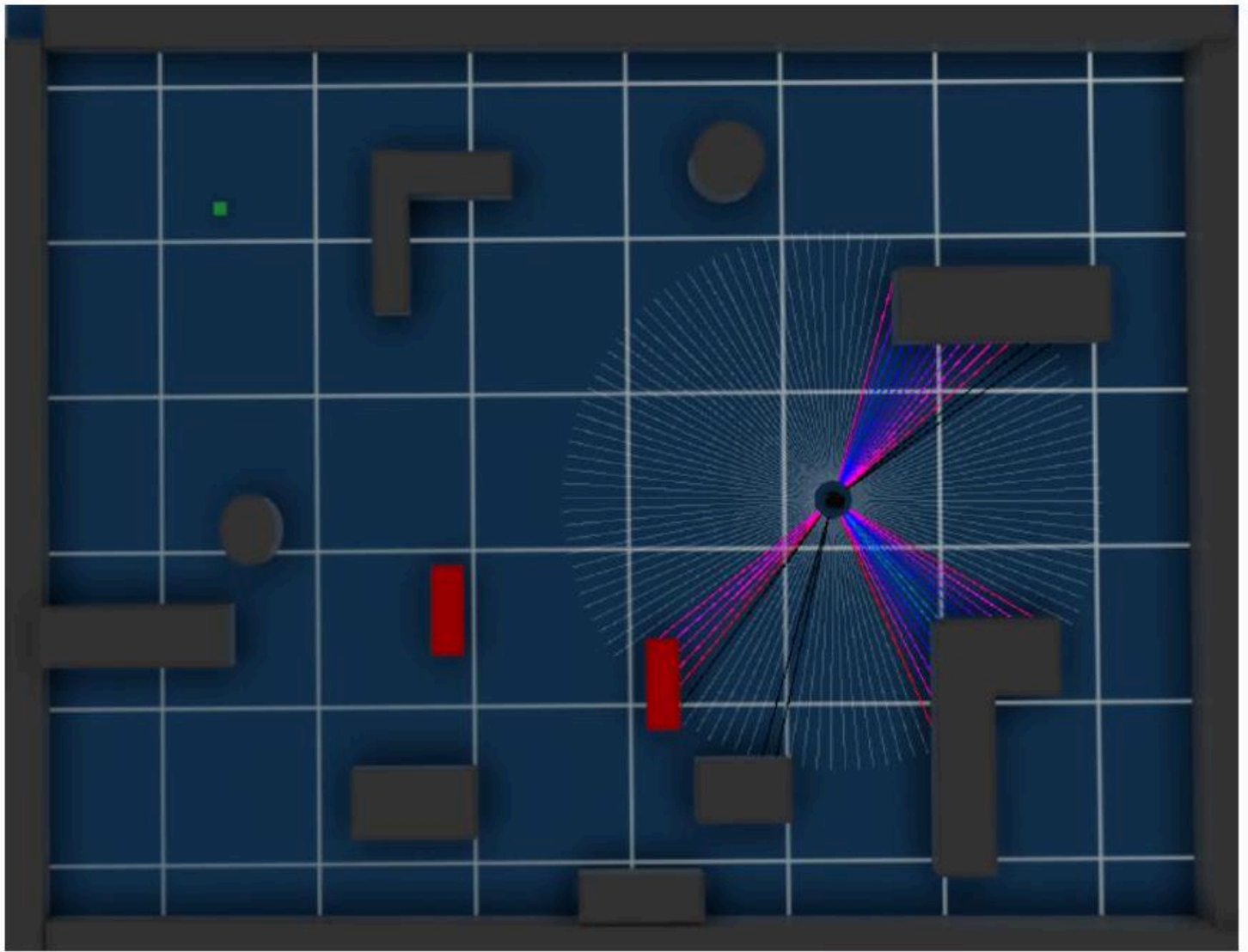


Figure 4: Fig. 4: Illustration of the robot and lidar scan in an Isaac environment with dynamic obstacles pictured in red.

圖 4：圖 4：機器人與光達掃描在 Isaac 環境中的示意圖，動態障礙物以紅色標示。

selects actions per its policy, while the Critic assesses the Actor's decisions. The Actor-Critic model utilizes a logarithmic standard deviation (logstd) for continuous action space, resulting in actions defined by a mean value with a fixed standard deviation. The network structure consists of a MultiLayer Perception (MLP) with 3 hidden layers of sizes [256, 128, 64]. Additionally, for training in the dynamic environment, we experimented with a Long Short-Term Memory (LSTM) layer of 128 hidden units after the input, followed by the MLP. As described earlier, these parameters are configured in the task and train config files. We train and test with two mobile robots, the Isaac built-in Jetbot, Figure 2a, and the Turtlebot3 robot imported from

根據其策略選擇動作，而評論者則評估執行者的決策。執行者-評論者模型使用對數標準差 (logstd) 進行連續動作空間，從而產生由具有固定標準差的平均值定義的動作。網路結構由一個多層感知器 (MLP) 組成，具有 3 個隱藏層，大小分別為 [256, 128, 64]。此外，為了在動態環境中進行訓練，我們在輸入之後嘗試使用一個具有 128 個隱藏單元的長短期記憶 (LSTM) 層，然後是 MLP。如前所述，這些參數在任務和訓練設定檔中進行配置。我們使用兩個移動機器人進行訓練和測試，即 Isaac 內建的 Jetbot (圖 2a) 和從

its URDF format, Figure 2b. When changing the robot, certain parameters in the initialization function must be adjusted, such as the wheel and speed settings in the DifferentialController class or the minimum and maximum lidar scan ranges.

其 URDF 格式匯入的 Turtlebot3 機器人 (圖 2b)。更換機器人時，必須調整初始化函數中的某些參數，例如 DifferentialController 類別中的車輪和速度設定，或最小和最大光達掃描範圍。

5 Experimental Results

5 實驗結果

5.1 Training In Isaac

5.1 在 Isaac 中訓練

The training results, depicted in Figure 3, show our experiments conducted in static and dynamic environments. In the static environment, the aforementioned MLP structure achieved a maximum of 73 rewards across 1000 episodes, signifying the successful accumulation of rewards for reaching the target in the shortest possible time. We employed the same network structure but added dynamic obstacles moving within the environment, red cubes in Fig4, which inherently complicated the training process. To further capture previous observations, we extended our training to include an LSTM layer followed by the same MLP layers. The training process in the dynamic environment shows limited promise, even when utilizing an LSTM layer. In fact, the robot fails to avoid obstacles and reach the target within 1000 episodes.

訓練結果如圖 3 所示，我們的實驗在靜態和動態環境中進行。在靜態環境中，上述 MLP 結構在 1000 個回合中獲得了最高 73 個獎勵，這表示成功累積了在最短時間內到達目標的獎勵。我們採用了相同的網路結構，但增加了在環境中移動的動態障礙物（圖 4 中的紅色立方體），這本質上使訓練過程複雜化。為了進一步捕捉先前的觀察結果，我們將訓練擴展到包括一個 LSTM 層，然後是相同的 MLP 層。動態環境中的訓練過程顯示出有限的前景，即使使用了 LSTM 層。事實上，機器人在 1000 個回合內未能避開障礙物並到達目標。

We adopted a curriculum learning approach for the dynamic environment to facilitate and accelerate the learning process, and enhance convergence. Curriculum learning is a training strategy that gradually increases the complexity of tasks or training samples. When performing an advanced navigation task, the initial step involves moving toward the target with associated rewards. Then, the environment's complexity can be enhanced by including static obstacles, and a penalty for collisions in the reward function. Finally, dynamic obstacles are added during the training process. In our case, we initiated training with the simpler task, the static environment. This allowed the agent to learn initial policies for moving towards the target, resulting in faster convergence. After 300 initial steps, we added dynamic objects and continued the training process in a more complex environment. The staged curriculum approach enabled smoother adaptation to the dynamic obstacles, ultimately enhancing the agent's performance and episode return.

我們針對動態環境採用了課程學習方法，以促進和加速學習過程，並增強收斂性。課程學習是一種逐步增加任務或訓練樣本複雜性的訓練策略。在執行進階導航任務時，初始步驟涉及朝著目標移動並獲得相關獎勵。然後，可以透過包含靜態障礙物和獎勵函數中碰撞的懲罰來增強環境的複雜性。最後，在訓練過程中添加動態障礙物。在我們的案例中，我們從較簡單的任務（靜態環境）開始訓練。這使得代理能夠學習朝著目標移動的初始策略，從而實現更快的收斂。在最初的 300 個步驟之後，我們添加了動態物體並在更複雜的環境中繼續訓練過程。分階段的課程方法使得代理能夠更順暢地適應動態障礙物，最終提高了代理的性能和回合回報。

5.2 Isaac Simulations

5.2 Isaac 模擬

Within the process of validating our model, a series of tests were conducted within the Isaac simulation environment, utilizing both Jetbot and Turtlebot3

在驗證我們模型的過程中，我們在 Isaac 模擬環境中進行了一系列測試，使用了 Jetbot 和 Turtlebot3 機器人。

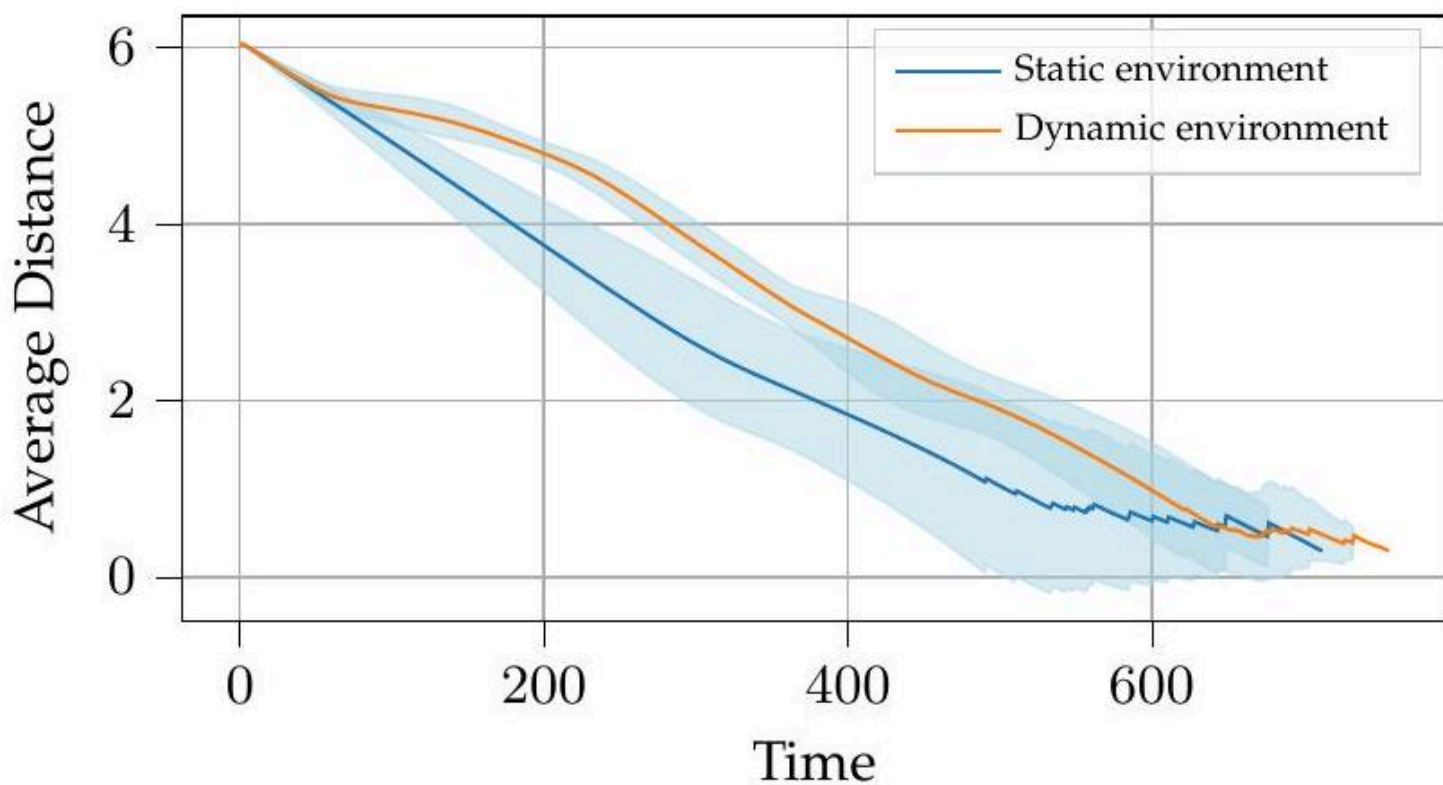


Figure 5: Fig. 5: Average and standard deviation of distance to the target as a function of time during tests using Jetbot in Isaac Sim with static and dynamic environments.

圖 5：圖 5：在 Isaac Sim 中使用 Jetbot 進行靜態和動態環境測試時，距離目標的平均值和標準差隨時間的變化。

robots. These tests involved setting various target positions to assess the model's adaptability. Figure 6a illustrates the Jetbot navigating through a novel environment, adeptly heading towards diverse target poses while avoiding obstacles along multiple paths. For Turtlebot3, Figure 6 b conveys the trajectory outcomes, where the proximity to obstacles is indicated by a gradient of colors, reflecting the robot's distance to the nearest object. This visual representation underscores the model's proficiency in maintaining a safe distance from obstacles, a critical aspect of autonomous navigation. Extending the assessment to dynamic settings, Figure 6c depicts robot's successful navigation past moving objects, effectively avoiding collisions. This performance is attributed to the model's curriculum-based training, as discussed in the preceding section.

這些測試涉及設定各種目標位置，以評估模型的可適應性。圖 6a 顯示了 Jetbot 在新環境中導航，熟練地朝著不同的目標姿態前進，同時避開多條路徑上的障礙物。對於 Turtlebot3，圖 6b 傳達了軌跡結果，其中與障礙物的接近程度由顏色梯度表示，反映了機器人與最近物體的距離。這種視覺表示強調了模型在與障礙物保持安全距離方面的熟練程度，這是自主導航的關鍵方面。將評估擴展到動態設置，圖 6c 描繪了機器人成功導航通過移動物體，有效避免碰撞。這種性能歸因於模型的課程式訓練，如前一節所述。

Finally, Figure 5 presents an aggregation of the robots' relative distances to a specified target from Figure 6c. compiled over 30 trials. This data encompasses scenarios with and without dynamic obstacles, providing a comprehensive view of the model's efficacy in variable environments.

最後，圖 5 呈現了圖 6c 中機器人與指定目標的相對距離的匯總，該數據是經過 30 次試驗編譯而成的。此數據包含有動態障礙物和無動態障礙物的情境，提供了模型在可變環境中有效性的全面視圖。

5.3 Gazebo Simulations

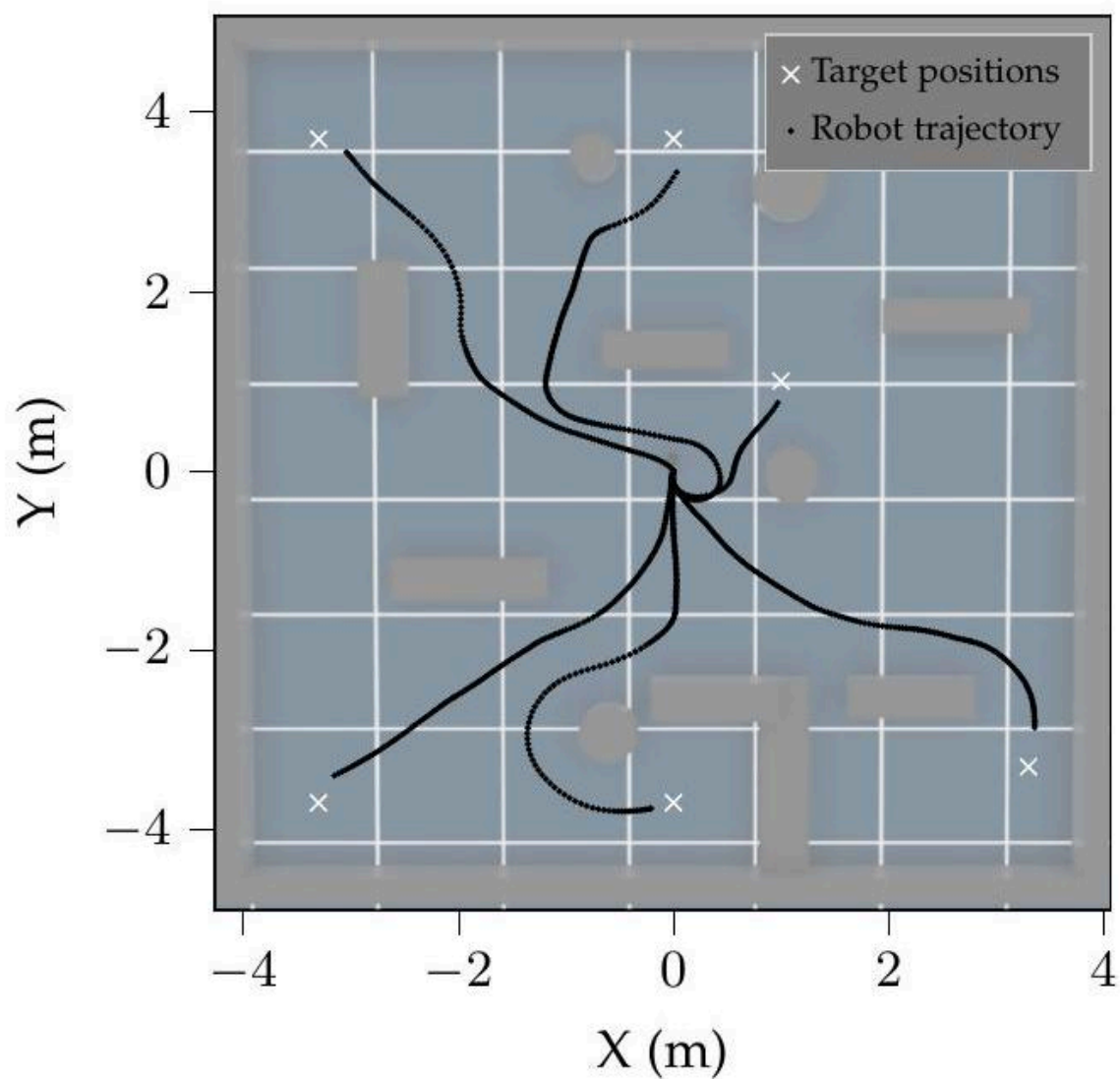
5.3 Gazebo 模擬

Prior to real-world deployment, the RL model was converted to an ONNX format and subjected to tests in various Gazebo environments using a ROS 2 node. We conducted a comparative analysis of the LSTM- and MLP-based RL models against Nav2 [20], the de facto ROS2 navigation stack, a sophisticated control system designed for autonomous robot navigation to a goal state based on the robot's current position, a map, and a target location. Figure 7 illustrates the comparative results in both static and dynamic settings.

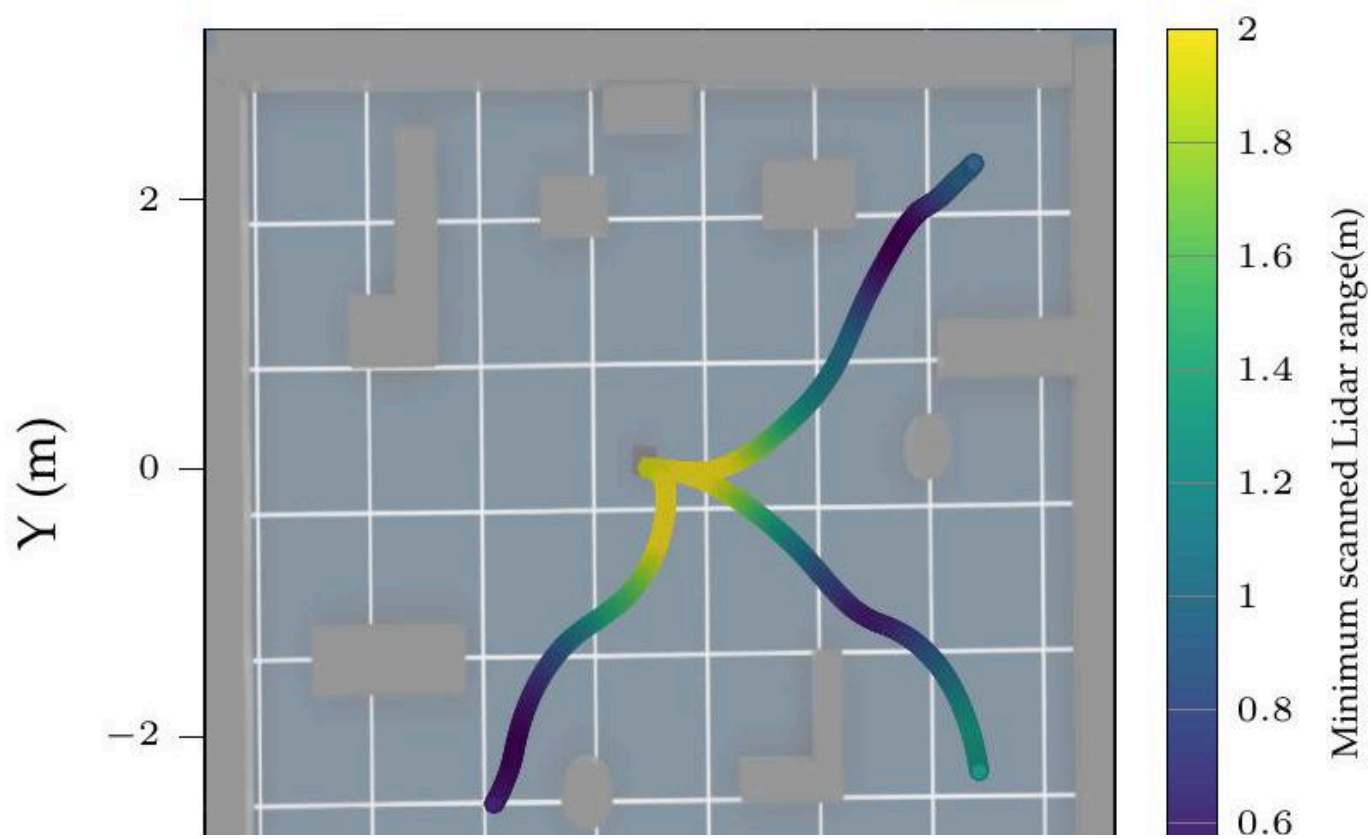
在實際部署之前，RL 模型被轉換為 ONNX 格式，並使用 ROS 2 節點在各種 Gazebo 環境中進行測試。我們對基於 LSTM 和 MLP 的 RL 模型與 Nav2 [20] 進行了比較分析，Nav2 是事實上的 ROS2 導航堆疊，一個複雜的控制系統，旨在根據機器人的當前位置、地圖和目標位置，實現自主機器人導航到目標狀態。圖 7 說明了在靜態和動態設定下的比較結果。

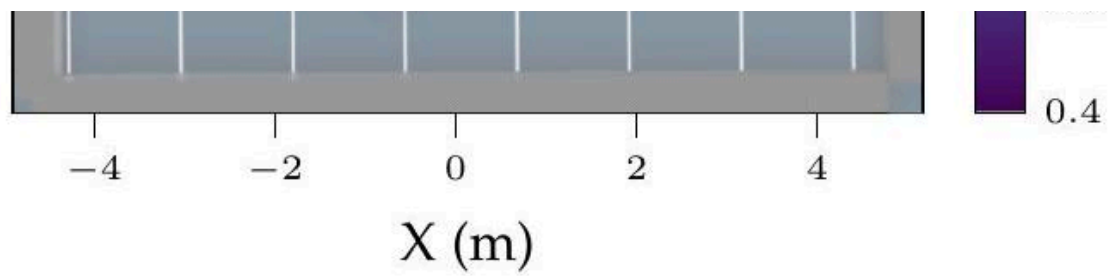
In the experiments, a mobile box was placed ahead of the robot, with its velocity adjusted be-

在實驗中，一個移動箱子被放置在機器人前方，其速度被調整為

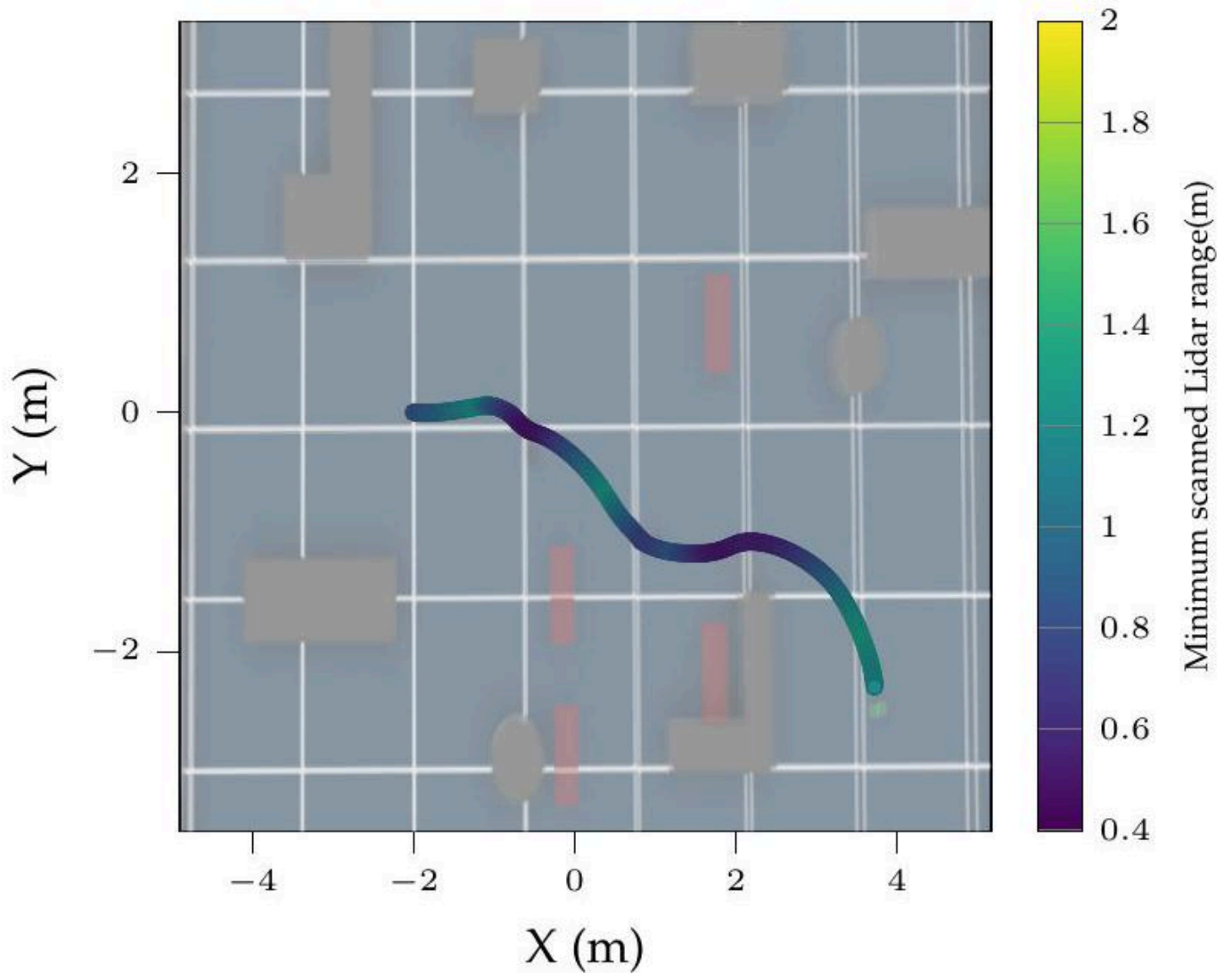


(a) Jetbot trajectory, different targets (static environment)





(b) Turtlebot3 trajectory, safety spectrum (static environment).



(c) Jetbot trajectory in dynamic environment.

Figure 6: Fig. 6: Qualitative evaluation performance with minimum lidar range throughout the trajectories in Isaac Sim.

圖 6：圖 6：在 Isaac Sim 中，沿著軌跡的最小雷射雷達範圍的定性評估性能。

tween 0.1 and 0.3 m/s across various trials to obstruct the robot's trajectory toward the designated target. Over 10 iterations of the same path, the distance-to-target distribution over time was comparable across all three methods (MLP-, LSTMbased RL, and Nav2) in static environments. This outcome is expected, as the Nav2 stack relies on a precomputed cost map, which can be less effective when encountering dynamic obstacles or sudden environmental changes, potentially

leading to mission failure.

在各種試驗中，機器人朝向指定目標的軌跡會被介於 0.1 和 0.3 m/s 之間的障礙物阻擋。在靜態環境中，相同路徑的 10 次迭代中，三種方法（基於 MLP、基於 LSTM 的 RL 和 Nav2）的距離目標分佈隨時間變化是可比較的。這個結果是預期的，因為 Nav2 堆疊依賴於預先計算的成本地圖，這在遇到動態障礙物或突然的環境變化時可能效果較差，可能導致任務失敗。

In contrast, the LSTM-based RL model, specifically trained to handle such scenarios, demonstrated superior performance. It consistently navigated around obstacles and avoided collisions, showcasing its robustness in dynamic conditions and its potential for reliable real-world applications. Meanwhile, the one-step training model using the MLP exhibited reduced performance in dynamic obstacle avoidance, underscoring its limited effectiveness for this task.

相較之下，基於 LSTM 的 RL 模型，特別是為處理此類情境而訓練的模型，展現了卓越的性能。它始終如一地繞過障礙物並避免碰撞，展示了其在動態條件下的穩健性及其在可靠的實際應用中的潛力。同時，使用 MLP 的一步訓練模型在動態避障方面的性能有所下降，突顯了其在該任務中的有限有效性。

5.4 Real-World Validation

5.4 實際驗證

The approach adopted in this research is adaptable to various mobile robotic systems. For the practical implementation and assessment of the RL navigation model, we employed the TurtleBot 4 Lite, which was equipped with an RPLIDAR A1M8, granting a complete 360-degree perspective, and interfaced with a Raspberry Pi 4B as the On-board Computer, Figure 8. The dimension of the TurtleBot 4 Lite is $342 \times 339 \times 192$ mm, with wheels measuring 72 mm in diameter. Its maximum safe mode linear velocity is 0.31 m/s. The LiDAR has a minimum detection distance of 0.15 m, with its range configured up to 2 m and a resolution of 3 degrees.

本研究採用的方法適用於各種移動機器人系統。為了實際實施和評估 RL 導航模型，我們使用了 TurtleBot 4 Lite，它配備了 RPLIDAR A1M8，可提供完整的 360 度視角，並與 Raspberry Pi 4B 作為機載電腦連接，如圖 8 所示。TurtleBot 4 Lite 的尺寸為 $342 \times 339 \times 192$ mm，車輪直徑為 72 毫米。其最大安全模式線性速度為 0.31 m/s。LiDAR 的最小檢測距離為 0.15 米，其範圍配置為 2 米，解析度為 3 度。

The refined model weights have been exported and integrated into a ROS2 Galactic node. The control system runs on the Raspberry Pi On-board Computer. The robot's positioning data was captured using an Optitrack Motion Capture system. To evaluate the system in real-world static environments, we designed test scenarios with obstacles of varying sizes and shapes, ensuring these differed from those used during training to assess generalization. Additionally, the robot's starting position was carefully chosen to align with the training setup, where it consistently began in relatively free space. This was necessary to avoid requiring additional training steps to account for diverse initial conditions. Figure 9 illustrates the robot's navigation paths in four distinct settings

經過優化的模型權重已匯出並整合到 ROS2 Galactic 節點中。控制系統運行在 Raspberry Pi 嵌入式電腦上。機器人的定位數據是使用 Optitrack 運動捕捉系統獲取的。為了評估系統在真實靜態環境中的表現，我們設計了包含不同大小和形狀障礙物的測試場景，確保這些障礙物與訓練期間使用的不同，以評估泛化能力。此外，機器人的起始位置經過精心選擇，與訓練設置保持一致，即它始終從相對空曠的空間開始。這是為了避免需要額外的訓練步驟來考慮多樣化的初始條件。圖 9 說明了機器人在四種不同環境中的導航路徑。

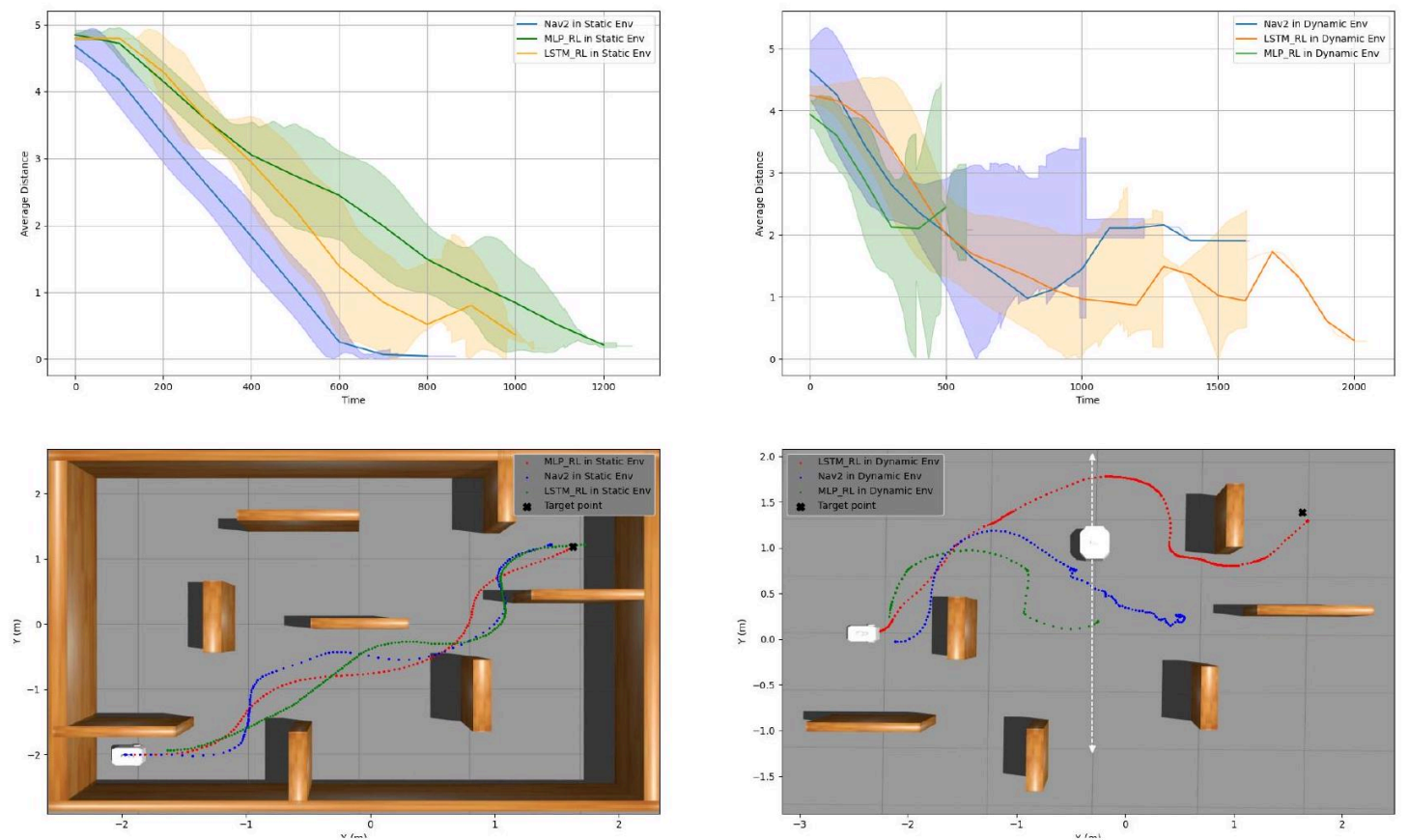


Figure 7: Fig. 7: Comparative performance analysis of the RL policy and Nav2 Stack in Gazebo.

圖 7：圖 7：RL 策略與 Nav2 Stack 在 Gazebo 中的比較性能分析。

Table 1: TABLE 1: Real-world performance statistics over 10 trials for 4 different experiments.

表 1：表 1：在 4 種不同實驗中，10 次試驗的真實世界性能統計數據。

	Task	Success	Min. lidar	Avg. linear	Dist. to
	任務	成功	最小雷射雷達	平均線性	距離
	time (s)	rate	range (m)	vel. (m/s)	target (m)
	時間 (秒)	率	範圍 (m)	速度 (m/s)	目標 (m)
Exp. 1					
實驗 1	39	10/10	0.43	0.15	4.69
Exp. 2					
實驗 2	41	10/10	0.39	0.15	4.70
Exp. 3					
實驗 3	53	8/10	0.40	0.13	5
Exp. 4					
實驗 4	52	7/10	0.25	0.14	4.73



Figure 8: Fig. 8: TurtleBot 4 Lite robot with 2D lidar used for the real-world experimental evaluation.

圖 8：圖 8：用於真實世界實驗評估的 TurtleBot 4 Lite 機器人，配備 2D 光達。

with varied obstacles. As shown in Figure 10, the robot's trajectories at different timestamps illustrate

在各種障礙物中。如圖 10 所示，機器人在不同時間戳的軌跡說明了

its movement while navigating towards the target. We conducted 10 trials for each test, and the bolded average performance metrics are presented in Table 1. The results from the successful trials indicate that the robots maintained a minimum distance of 25 cm from obstacles, aligning with the training threshold. This safety margin can be adjusted to suit different robot dimensions and configurations. Similarly, the linear velocity parameter is flexible and can be tailored as required.

其在導航至目標時的移動。我們對每個測試進行了 10 次試驗，粗體顯示的平均性能指標呈現在表 1 中。成功試驗的結果表明，機器人與障礙物保持了至少 25 公分的距離，這與訓練閾值一致。此安全裕度可根據不同的機器人尺寸和配置進行調整。同樣地，線性速度參數是靈活的，可以根據需要進行調整。

We expanded our experimentation to include dynamic obstacles-specifically, people-that obstruct the robot's path toward the target. The robot's trajectories, both without obstacles and with dynamic obstacles, are illustrated in Figure 11. To

highlight the robot's avoidance behavior, we marked the positions of the robot and obstacle every 15

我們擴大了實驗範圍，將動態障礙物（特別是人）納入其中，這些障礙物會阻礙機器人通往目標的路徑。圖 11 顯示了機器人在沒有障礙物和有動態障礙物兩種情況下的軌跡。為了突顯機器人的避障行為，我們每 15 個單位標記了機器人和障礙物的位置。

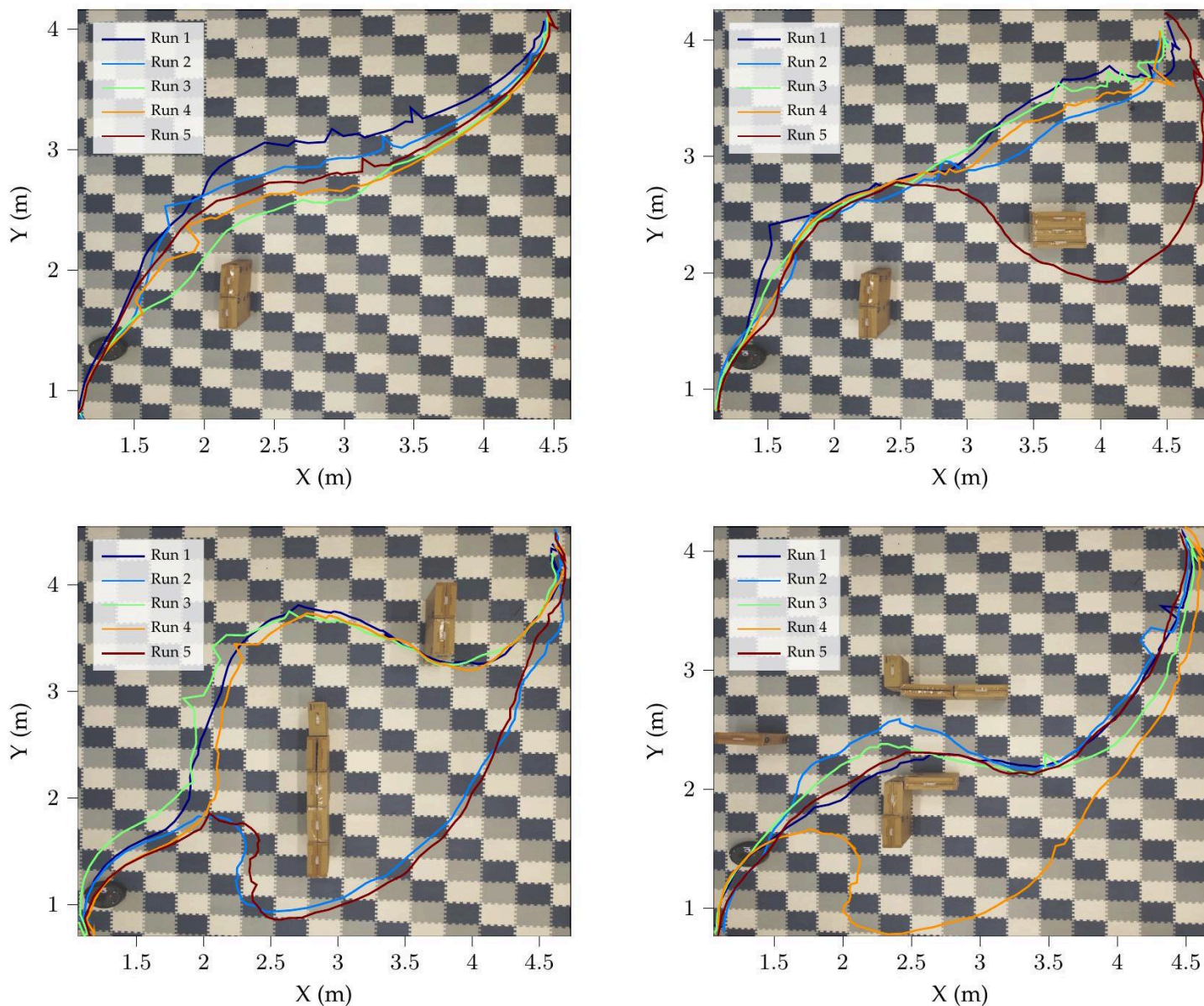


Figure 9: Fig. 9: Navigation in Various Real-World Environments

圖 9：圖 9：在各種真實世界環境中的導航

seconds, showing when the robot adjusts its path. The individuals moved unpredictably in front of the robot, partially obstructing its direct path toward the target. At times, the person would suddenly appear in the robot's trajectory, while in other instances, they moved alongside the robot, attempting to obstruct its path. This random movement pattern was designed to simulate real-world scenarios with a priori unpredictable dynamic obstacles.

秒，顯示機器人何時調整其路徑。這些人會不可預測地在機器人前方移動，部分阻礙了機器人通往目標的直接路徑。有時，人會突然出現在機器人的軌跡中，而在其他情況下，他們會與機器人並行移動，試圖阻礙其路徑。這種隨機移動模式旨在模擬具有先驗不可預測動態障礙物的真實世界場景。

While curriculum learning improves performance in dynamic environments, models often struggle with tasks beyond their training conditions in dynamic environments. Handling diverse obstacle sizes, shapes, directions, and speeds requires

carefully tuned reward functions and additional training stages to maintain robust performance.

儘管課程學習可以提高動態環境中的性能，但模型在動態環境中往往難以應對超出其訓練條件的任務。處理不同大小、形狀、方向和速度的障礙物需要仔細調整獎勵函數和額外的訓練階段，以保持穩健的性能。

6 DISCUSSION

6 討論

We tested the model in various static environments across simulators and the real world, achieving promising results. However, its generalization and robustness faced challenges in dynamic real-world settings. The presence of diverse conditions and noise-such as changes in the size, shape, speed, and direction of dynamic obstacles-highlighted the need for further adjustments. To improve performance in such scenarios, retraining, progressive training, and potential modifications to the reward function may be necessary. Additionally, real-world test feedback led to fine-tuning parameters, such as adjusting the LiDAR sampling process to account for narrower dynamic obstacles. These adjustments required either retraining from the baseline or more training to refine the reward function. To enable safe and effective navigation through dynamic obstacles such as humans, incorporating a specialized reward

我們在模擬器和現實世界的各種靜態環境中測試了模型，取得了可喜的成果。然而，其泛化能力和穩健性在動態現實世界環境中面臨挑戰。多樣的條件和噪音（例如動態障礙物的大小、形狀、速度和方向的變化）的存在，凸顯了進一步調整的必要性。為了改善此類情境下的性能，可能需要重新訓練、漸進式訓練以及對獎勵函數進行潛在修改。此外，現實世界測試回饋導致了參數的微調，例如調整 LiDAR 採樣過程以考慮更窄的動態障礙物。這些調整需要從基準線重新訓練或進行更多訓練以完善獎勵函數。為了實現安全有效地穿越人類等動態障礙物，需要納入專門的獎勵

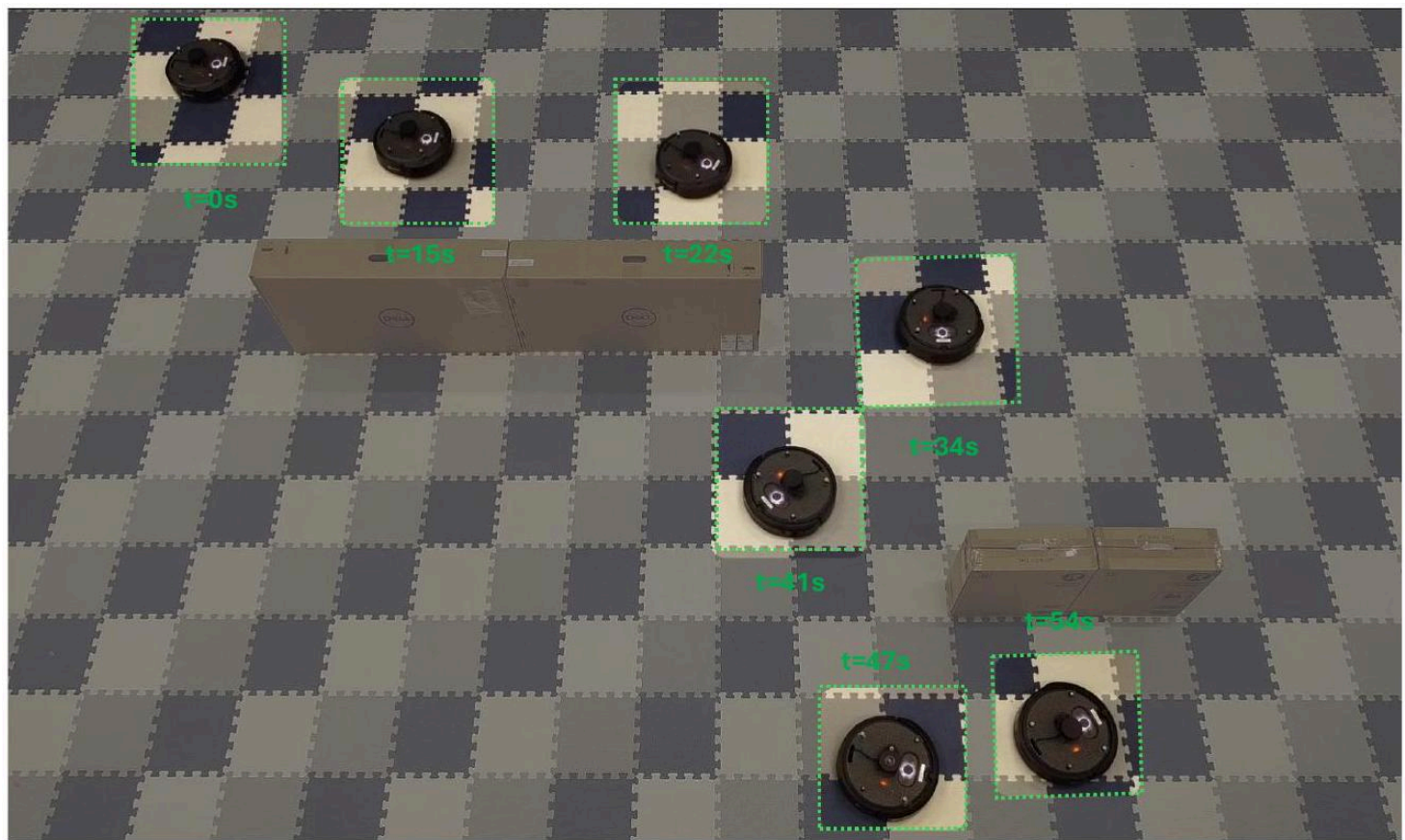


Figure 10: Fig. 10: Robot trajectories with timestamps showing obstacle avoidance and navigating towards the target.

圖 10：圖 10：帶有時間戳的機器人軌跡，顯示避障和導航至目標。

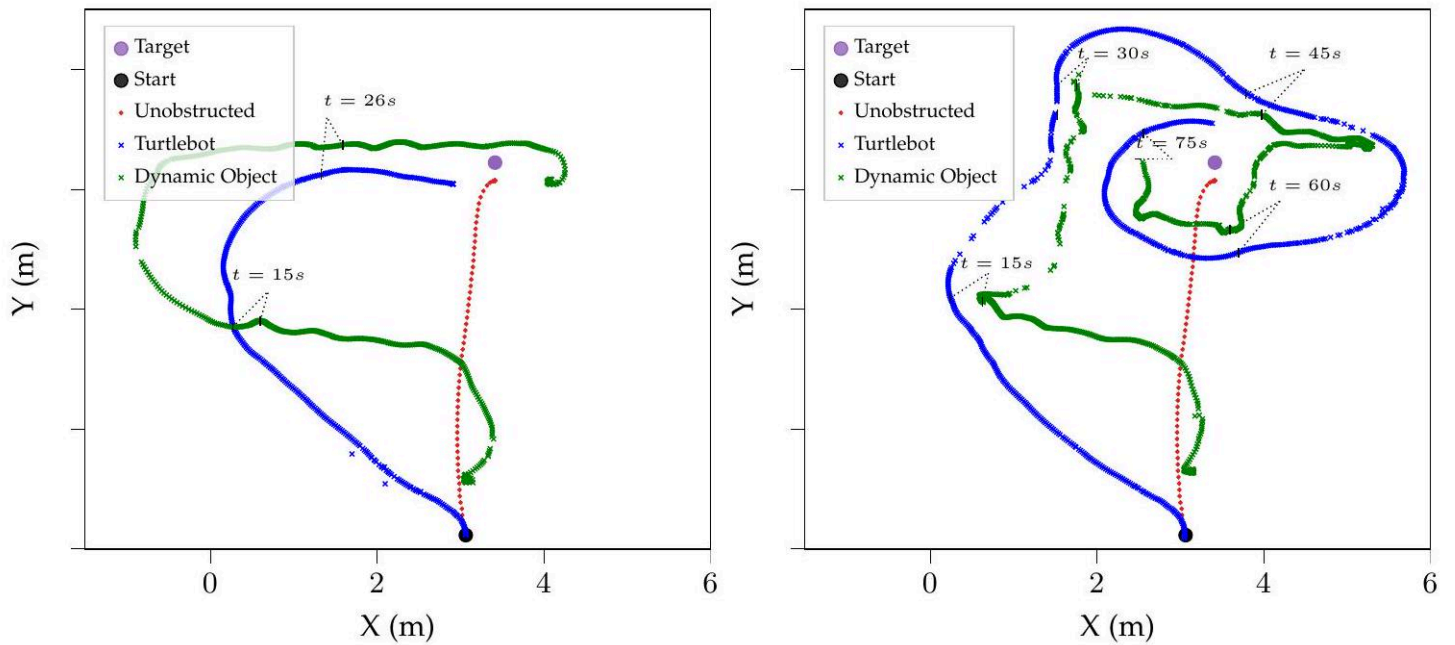


Figure 11: Fig. 11: Trajectories of Turtlebot in the presence of a dynamic obstacle in real-world experiments.

圖 11：圖 11：Turtlebot 在現實世界實驗中存在動態障礙物時的軌跡。

function term, such as a social-safety zone, may be essential for fostering human-aware navigation. This approach can be adapted and generalized for other specific environments requiring tailored modifications.

功能術語，例如社交安全區，對於促進人類感知導航可能至關重要。這種方法可以針對需要客製化修改的其他特定環境進行調整和推廣。

7 Conclusion

7 結論

Throughout this article, we have described the process of setting up a training workflow for a RL policy for mobile robot navigation in Isaac Sim. We covered the key steps in defining the robot model, and the training environment and RL task. Additionally, we discuss important aspects in terms of hyperparameter tuning from the perspective of

在本文中，我們描述了在 Isaac Sim 中為行動機器人導航設定 RL 策略訓練工作流程的過程。我們涵蓋了定義機器人模型、訓練環境和 RL 任務的關鍵步驟。此外，我們從

both the training setup and the actual policy model. Finally, we describe the workflow to enable the transfer from simulation to reality, with examples and ROS 2 node templates.

訓練設定和實際策略模型的角度討論了超參數調整的重要方面。最後，我們描述了實現從模擬到現實轉移的工作流程，並提供了範例和 ROS 2 節點模板。

To demonstrate the effectiveness and usability of such RL policies for real-world deployments, we also analyze the performance both quantitatively and qualitatively in simulation (Isaac Sim and Gazebo) and real-world experiments. We use Nav2, the de-facto standard ROS2 navigation stack, as a benchmark in a subset of the simulations.

為了展示此類強化學習策略在實際部署中的有效性和可用性，我們還在模擬（Isaac Sim 和 Gazebo）和實際實驗中，對其性能進行了定量和定性分析。我們使用 Nav2（事實上的標準 ROS2 導航堆疊）作為部分模擬的基準。

The experimental results convey that state-of-the-art performance can be obtained by training fully in the simulation environment. This opens the

實驗結果表明，透過在模擬環境中進行全面訓練，可以獲得最先進的性能。這為

door to quick deployment of new robots with end-to-end RL-based control, where both perception, trajectory planning and tracking are encapsulated in a single process. While this does not necessarily offer the best fine-tuned performance, we believe this to be a step towards, e.g., low-code applications.

新機器人的快速部署打開了大門，這些機器人採用端到端基於強化學習的控制，其中感知、軌跡規劃和追蹤都封裝在一個單一的過程中。雖然這不一定能提供最佳的微調性能，但我們相信這是邁向例如低程式碼應用程式的一步。

Overall, this work aims to discuss the different possible approaches to RL-based local navigation and obstacle avoidance, beyond the specific approaches and use-cases widely showcased in the literature. This has been presented in an instructional style, but also covering new experimental results. Importantly, we believe this work fills a gap in the literature in terms of the introduction of generic sim-to-real workflows and a generalizable approach to RL navigation in mobile robotics.

總體而言，這項工作旨在討論基於強化學習的局部導航和避障的不同可能方法，超越了文獻中廣泛展示的特定方法和用例。這以教學風格呈現，但也涵蓋了新的實驗結果。重要的是，我們相信這項工作填補了文獻中在通用模擬到現實工作流程和行動機器人中強化學習導航的通用方法方面的空白。

Acknowledgments

致謝

This work was supported by the R3Swarms project funded by the Technology Innovation Institute (TII).

這項工作由技術創新研究所（TII）資助的 R3Swarms 專案支持。

References

參考文獻

- [1] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, “Rloc: Terrain-aware legged locomotion using reinforcement learning and optimal control,” *IEEE Transactions on Robotics*, vol. 38, no. 5, 2022.
- [2] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, “Learning robust autonomous navigation and locomotion for wheeled-legged robots,” *Science Robotics*, vol. 9, no. 89, 2024.
- [3] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, “Reaching the limit in autonomous racing: Optimal control versus reinforcement learning,” *Science Robotics*, vol. 8, no. 82, 2023.
- [4] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, “Real-world humanoid locomotion with reinforcement learning,” *Science Robotics*, vol. 9, 2024.
- [5] P. Egli and M. Hutter, “Towards rl-based hydraulic excavator automation,” in *IEEE IROS*, 2020.
- [6] D. Han, B. Mulyana, V. Stankovic, and S. Cheng, “A survey on deep reinforcement learning algorithms for robotic manipulation,” *MDPI Sensors*, vol. 23, no. 7, 2023.
- [7] G. B. Margolis, G. Yang, K. Paigwar, T. Chen, and P. Agrawal, “Rapid locomotion via reinforcement learning,” *The International Journal of Robotics Research*, 2024.
- [8] T.-Y. Yang, T. Zhang, L. Luu, S. Ha, J. Tan, and W. Yu, “Safe reinforcement learning for legged locomotion,” in *IEEE IROS*, 2022.
- [9] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, et al., “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE Robotics and Automation Letters*, 2023.
- [10] W. Zhao, E.-A. Rantala, J. Pajarinen, and J. P. Queralta, “Less is more: Robust robot learning via partially observable multi-agent reinforcement learning,” *arXiv*, 2023.
- [11] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: a survey,” in *IEEE SSCI*, 2020.
- [12] S. Aradi, “Survey of deep reinforcement learning for motion planning of autonomous vehicles,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, 2020.
- [13] J. Choi, G. Lee, and C. Lee, “Reinforcement learningbased dynamic obstacle avoidance and integration of path planning,” *Intelligent Service Robotics*, vol. 14, 2021.
- [14] W. Li, M. Yue, J. Shangguan, and Y. Jin, “Navigation of mobile robots based on deep reinforcement learning: Reward function optimization and knowledge transfer,” *Intl. Journal of Control, Automation and Systems*, 2023.

[14] W. Li, M. Yue, J. Shangguan, and Y. Jin, 「基於深度強化學習的移動機器人導航：獎勵函數優化與知識轉移」,《國際控制、自動化與系統期刊》, 2023。

[15] H. Surmann, C. Jestel, R. Marchel, F. Musberg, H. Elhadj, and M. Ardani, “Deep reinforcement learning for real autonomous mobile robot navigation in indoor environments,” arXiv, 2020.

[15] H. Surmann, C. Jestel, R. Marchel, F. Musberg, H. Elhadj, and M. Ardani, 「用於室內環境中真實自主移動機器人導航的深度強化學習」, arXiv , 2020。

[16] J. Gao, W. Ye, J. Guo, and Z. Li, “Deep reinforcement learning for indoor mobile robot path planning,” MDPI Sensors, vol. 20, no. 19, 2020.

[16] J. Gao, W. Ye, J. Guo, and Z. Li, 「用於室內移動機器人路徑規劃的深度強化學習」,《MDPI 感測器》, 第 20 卷, 第 19 期, 2020。

[17] N. Ü. Akmandor, H. Li, G. Lvov, E. Dusel, and T. Padir, “Deep reinforcement learning based robot navigation in dynamic environments using occupancy values of motion primitives,” in IEEE IROS, 2022.

[17] N. Ü. Akmandor, H. Li, G. Lvov, E. Dusel, and T. Padir, 「基於深度強化學習的機器人在動態環境中利用運動基元佔用值進行導航」, 收錄於 IEEE IROS , 2022。

[18] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” Nature, vol. 620, no. 7976, 2023.

[19] I. Kim, S. H. Nengroo, and D. Har, “Reinforcement learning for navigation of mobile robot with lidar,” in IEEE ICECA Technology, IEEE, 2021.

[20] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, “From the desks of ros maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2,” Robotics and Autonomous Systems, vol. 168, 2023.

<https://isaac-sim.github.io/IsaacLab/main/index.html>